

# OE-EC704A

## Web Technology

---

### Combined Module Notes

Modules 1 – 10

Semester VII • ECE • 3 Credits

CGEC | B.Tech ECE (2023–27)

#### Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

**Biraj Sarkar**

IN ASSOCIATION WITH

MAKAUT Wingman • MAKAUT Future Minds

**Cooch Behar Government Engineering College**

*Exam-Ready Notes*

## Contents

---

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>Module 1: Introduction to Web Development Basics</b> | <b>5</b>  |
| <b>1 Introduction to Web Development Basics</b>         | <b>5</b>  |
| 1.1 HTML vs. XML                                        | 5         |
| <b>2 HTML Basics and Document Structure</b>             | <b>6</b>  |
| 2.1 Standard Skeleton of an HTML5 Document              | 6         |
| 2.2 Block-level vs. Inline Elements                     | 7         |
| 2.3 Essential HTML Components                           | 7         |
| <b>3 CSS (Cascading Style Sheets)</b>                   | <b>8</b>  |
| 3.1 Types of CSS Integration                            | 8         |
| 3.2 CSS Selectors                                       | 8         |
| 3.3 The CSS Box Model                                   | 9         |
| <b>Module 2: Java Execution Environment</b>             | <b>13</b> |
| <b>1 Java Execution Environment</b>                     | <b>13</b> |
| 1.1 JDK vs. JRE vs. JVM                                 | 13        |
| <b>2 Java Characteristics and Features</b>              | <b>14</b> |
| <b>3 Object-Oriented Programming (OOP) Concepts</b>     | <b>15</b> |
| 3.1 The Four Pillars of OOP                             | 15        |
| <b>4 Structure of a Simple Java Program</b>             | <b>16</b> |
| 4.1 Standard Hello World Program                        | 16        |
| 4.2 Key Elements Explained                              | 16        |
| 4.3 Compiling and Running Java Programs                 | 16        |
| <b>5 Data Types, Operators, and Expressions</b>         | <b>17</b> |
| 5.1 Primitive vs. Reference Data Types                  | 17        |
| 5.2 Memory Allocation Model                             | 17        |
| 5.3 Operators and Expressions                           | 18        |
| <b>6 Control Flow and Branching</b>                     | <b>18</b> |
| <b>Module 3: Class Fundamentals and Objects</b>         | <b>22</b> |
| <b>1 Class Fundamentals and Objects</b>                 | <b>22</b> |
| 1.1 Constructors in Java                                | 22        |
| 1.2 Argument Passing in Java                            | 23        |
| 1.3 The <code>static</code> Keyword                     | 23        |
| 1.4 Nested and Inner Classes                            | 23        |
| <b>2 Inheritance and Reuse Models</b>                   | <b>23</b> |
| 2.1 Composition vs. Inheritance                         | 24        |
| <b>3 Polymorphism and Runtime Resolution</b>            | <b>24</b> |

|     |                                                               |    |
|-----|---------------------------------------------------------------|----|
| 3.1 | Method Overloading vs. Overriding . . . . .                   | 24 |
| 3.2 | Dynamic Method Dispatch . . . . .                             | 25 |
| 3.3 | Abstract Classes and the <code>final</code> Keyword . . . . . | 25 |
| 3.4 | Access Modifiers . . . . .                                    | 25 |

## Module 4: Interfaces 29

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Interfaces . . . . .</b>                   | <b>29</b> |
| 1.1      | Syntax and Implementation . . . . .           | 29        |
| 1.2      | Interface Additions in Java 8 and 9 . . . . . | 29        |
| <b>2</b> | <b>Packages . . . . .</b>                     | <b>30</b> |
| <b>3</b> | <b>Access Control Mechanism . . . . .</b>     | <b>30</b> |
| <b>4</b> | <b>Dynamic Method Lookup . . . . .</b>        | <b>31</b> |

## Module 5: Java Exception Handling Mechanism 35

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Java Exception Handling Mechanism . . . . .</b>                        | <b>35</b> |
| 1.1      | Throwable Hierarchy . . . . .                                             | 35        |
| 1.2      | Checked vs. Unchecked Exceptions . . . . .                                | 36        |
| <b>2</b> | <b>Exception Keywords &amp; Handling Structures . . . . .</b>             | <b>36</b> |
| 2.1      | Try-Catch-Finally Blocks . . . . .                                        | 36        |
| 2.2      | Throw vs. Throws . . . . .                                                | 36        |
| 2.3      | Try-with-Resources and the <code>AutoCloseable</code> Interface . . . . . | 36        |
| 2.4      | User-Defined / Custom Exceptions . . . . .                                | 37        |

## Module 6: String Basics and Immutability 41

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>String Basics and Immutability . . . . .</b>   | <b>41</b> |
| 1.1      | String Constant Pool (SCP) . . . . .              | 41        |
| <b>2</b> | <b>String vs. StringBuffer . . . . .</b>          | <b>42</b> |
| <b>3</b> | <b>Constructors and Core Operations . . . . .</b> | <b>42</b> |
| 3.1      | String Constructors . . . . .                     | 42        |
| 3.2      | Core String Operations . . . . .                  | 42        |
| <b>4</b> | <b>Utility Conversions . . . . .</b>              | <b>43</b> |
| <b>5</b> | <b>StringBuffer Operations . . . . .</b>          | <b>43</b> |

## Module 7: Java I/O Stream Basics 47

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Java I/O Stream Basics . . . . .</b>                   | <b>47</b> |
| 1.1      | Byte Stream vs. Character Stream . . . . .                | 47        |
| <b>2</b> | <b>Console I/O Operations . . . . .</b>                   | <b>47</b> |
| 2.1      | Reading Console via <code>BufferedReader</code> . . . . . | 48        |
| 2.2      | Reading Console via <code>Scanner</code> . . . . .        | 48        |
| <b>3</b> | <b>File I/O Operations . . . . .</b>                      | <b>48</b> |

|     |                                                |    |
|-----|------------------------------------------------|----|
| 3.1 | FileInputStream and FileOutputStream . . . . . | 48 |
| 3.2 | FileReader and FileWriter . . . . .            | 48 |
| 3.3 | Buffered Stream Wrapper Pattern . . . . .      | 48 |

## **Module 8: Java Collections Framework Overview** 53

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>1</b> | <b>Java Collections Framework Overview . . . . .</b> | <b>53</b> |
| 1.1      | Core Interfaces . . . . .                            | 53        |
| <b>2</b> | <b>ArrayList vs. LinkedList . . . . .</b>            | <b>54</b> |
| <b>3</b> | <b>Accessing and Iterating Collections . . . . .</b> | <b>54</b> |
| 3.1      | The Iterator Interface . . . . .                     | 54        |
| 3.2      | The For-Each Statement . . . . .                     | 55        |

## **Module 9: Introduction to Applets** 59

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction to Applets . . . . .</b>              | <b>59</b> |
| 1.1      | Applet Lifecycle & Execution . . . . .                | 59        |
| <b>2</b> | <b>Applet Display and Status Window . . . . .</b>     | <b>60</b> |
| <b>3</b> | <b>HTML Embedding and Parameter Passing . . . . .</b> | <b>60</b> |
| 3.1      | HTML Applet Tag Syntax . . . . .                      | 60        |
| 3.2      | Parameter Retrieval via getParameter() . . . . .      | 60        |
| 3.3      | getCodeBase() and getDocumentBase() . . . . .         | 61        |

## **Module 10: Delegation Event Model** 65

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Delegation Event Model . . . . .</b>      | <b>65</b> |
| 1.1      | Common Event Classes & Listeners . . . . .   | 65        |
| <b>2</b> | <b>Adapter Classes . . . . .</b>             | <b>66</b> |
| <b>3</b> | <b>Inner and Anonymous Classes . . . . .</b> | <b>66</b> |
| <b>4</b> | <b>AWT Controls . . . . .</b>                | <b>66</b> |
| <b>5</b> | <b>Layout Managers . . . . .</b>             | <b>67</b> |

# Module 1

## Introduction to Web Development Basics

---

### 1. Introduction to Web Development Basics

---

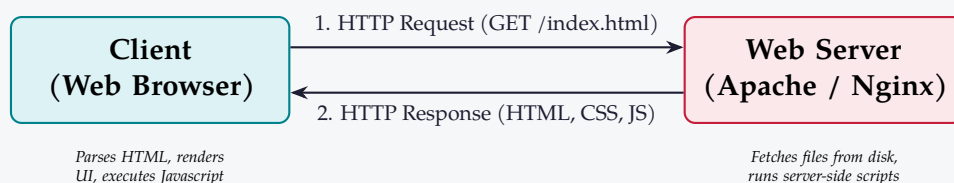
Modern web development relies on a client-server architecture. Web pages are hosted on web servers and requested by clients (web browsers) via HTTP/HTTPS protocols.

#### Client-Server Web Architecture

The **Client-Server Model** is a distributed application structure that partitions tasks between resource providers (**servers**) and service requesters (**clients**). In web technology:

- **Client (Web Browser):** Sends HTTP requests, parses HTML/CSS/JS, and renders the user interface.
- **Server (Web Server):** Listens for incoming requests on ports 80 (HTTP) or 443 (HTTPS), processes logic, and returns the requested resources.

#### Web Architecture: Client-Server Flow



#### 1.1 HTML vs. XML

HTML and XML are both markup languages based on SGML, but they serve fundamentally different purposes:

| Feature           | HTML (HyperText Markup Language)            | XML (eXtensible Markup Language)               |
|-------------------|---------------------------------------------|------------------------------------------------|
| Primary Purpose   | Displaying and formatting data to users.    | Transporting, storing, and structuring data.   |
| Tags              | Predefined tags (e.g., <html>, <p>).        | Custom/user-defined tags based on data.        |
| Syntax Strictness | Forgiving (errors don't crash rendering).   | Extremely strict (well-formed check required). |
| Case Sensitivity  | Case-insensitive (<DIV> and <div>).         | Strictly case-sensitive.                       |
| Closing Tags      | Optional for some tags (e.g., <img>, <br>). | Mandatory for all tags.                        |

## 2. HTML Basics and Document Structure

### HTML — HyperText Markup Language

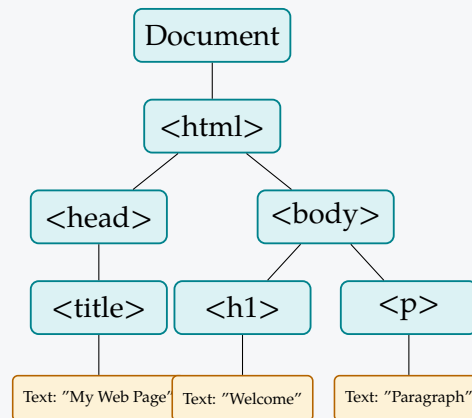
**HTML** is the standard markup language used to create the structure of web pages. It uses a set of hierarchical elements represented by tags to organize headings, paragraphs, tables, images, and other media elements.

### 2.1 Standard Skeleton of an HTML5 Document

Every HTML document must follow a standard structural hierarchy:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>My Web Page</title>
  </head>
  <body>
    <h1>Welcome to Web Technology</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

## HTML Document Object Model (DOM) Tree



## 2.2 Block-level vs. Inline Elements

HTML elements are categorized based on their default layout behaviors:

| Feature           | Block-level Elements                                                                                                                                                                   | Inline Elements                                                                                                                                                    |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Layout Behavior   | Starts on a new line and takes up full width.                                                                                                                                          | Stays on the same line and occupies only needed width.                                                                                                             |
| Margins & Padding | Fully respected on all four sides.                                                                                                                                                     | Top and bottom margins are ignored.                                                                                                                                |
| Containment       | Can contain block or inline elements.                                                                                                                                                  | Can only contain inline elements / text.                                                                                                                           |
| Examples          | <code>&lt;div&gt;</code> , <code>&lt;p&gt;</code> , <code>&lt;h1&gt;</code> – <code>&lt;h6&gt;</code> , <code>&lt;ul&gt;</code> , <code>&lt;ol&gt;</code> , <code>&lt;table&gt;</code> | <code>&lt;span&gt;</code> , <code>&lt;a&gt;</code> , <code>&lt;strong&gt;</code> , <code>&lt;em&gt;</code> , <code>&lt;img&gt;</code> , <code>&lt;input&gt;</code> |

## 2.3 Essential HTML Components

1. Lists: HTML supports three list types:

- **Unordered List (`<ul>`):** Bulleted list items using the `<li>` tag.
- **Ordered List (`<ol>`):** Numbered list items with customizable types (numeric, alphabetic, roman).
- **Description List (`<dl>`):** Pairs of terms (`<dt>`) and descriptions (`<dd>`).

2. Links (Anchors `<a>`): Used for navigating between pages or sections:

- **Absolute Link:** Links to a fully qualified URL (e.g., `href="https://google.com"`).
- **Relative Link:** Links to a local resource relative to the file path (e.g., `href="about.html"`).
- **Email Link:** Triggers the client's email client using the mailto protocol (e.g., `href="mailto:test@test.com"`).
- **Anchor Targets:** Uses `target="_blank"` to open in a new tab or `target="_self"` for the same frame.

3. Tables (`<table>`): Formats data in rows and columns:

- **Rowspan:** Extends a cell vertically across multiple rows (e.g., `<td rowspan="2">`).
- **Colspan:** Extends a cell horizontally across multiple columns (e.g., `<td colspan="3">`).

4. **Forms (<form>):** Captures and submits user inputs to a server. Key attributes:

- **action:** The URL of the server-side endpoint that handles the submitted form data.
- **method:** The HTTP method used to send data (GET or POST).
- **Input Types:** Text, password, radio buttons, checkboxes, email, submit, file, number.

5. **Frames (<frameset>, <frame>, <iframe>):** Used to split the browser window into multiple independent document spaces:

- **Traditional Frameset (<frameset>):** Replaces the `<body>` tag in HTML4 to partition the screen layout into rows or columns (e.g., `<frameset cols="25%, 75%">`). Each partition is defined using a `<frame src="...">` tag. *Note: Deprecated in HTML5 due to poor accessibility, bookmarking difficulties, and bad SEO indexing.*
- **Inline Frames (<iframe>):** Used inside a standard `<body>` tag to embed another HTML page within the current page (e.g., `<iframe src="inner.html" width="300" height="200"></iframe>`). Widely used for embedding maps, videos, and third-party widgets.

### 3. CSS (Cascading Style Sheets)

#### CSS — Cascading Style Sheets

CSS is a style sheet language used to describe the presentation, design, formatting, and layout of a document written in HTML. It separates structure from style, allowing consistent look-and-feel across multiple web pages.

#### 3.1 Types of CSS Integration

| CSS Type     | Syntax Example                                              | Advantages                                           | Disadvantages                            |
|--------------|-------------------------------------------------------------|------------------------------------------------------|------------------------------------------|
| Inline CSS   | <code>&lt;p style="color:red;"&gt;</code>                   | High priority; quick overrides.                      | Messy; violates separation of concerns.  |
| Internal CSS | <code>&lt;style&gt; p { color:red; } &lt;/style&gt;</code>  | Contained inside single page; no extra HTTP request. | Reusability limit across multiple pages. |
| External CSS | <code>&lt;link rel="stylesheet" href="style.css"&gt;</code> | High reuse; caching support; clean separation.       | Requires extra network HTTP request.     |

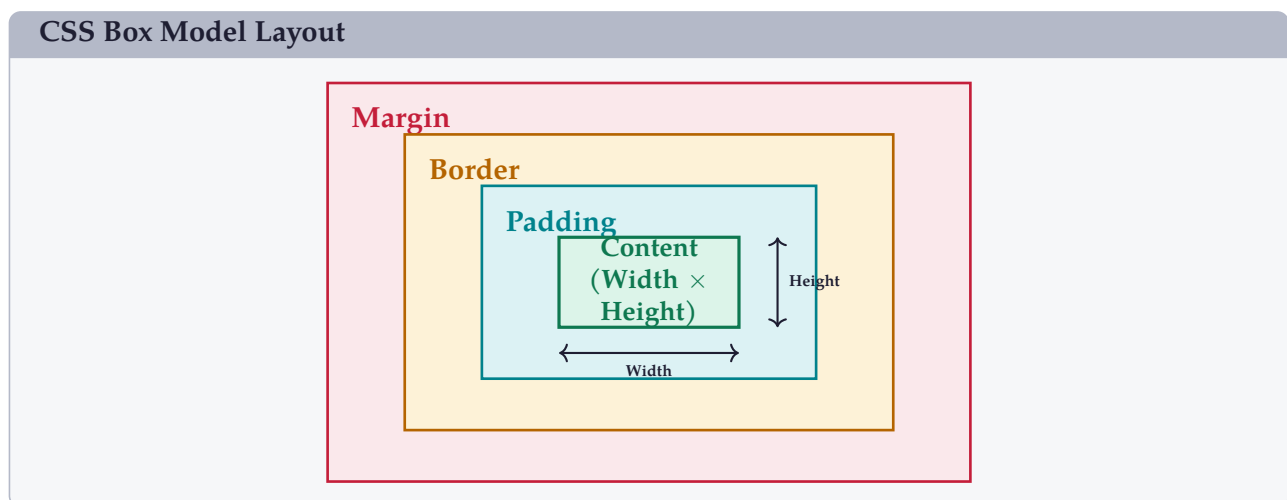
#### 3.2 CSS Selectors

CSS selectors target specific HTML elements for styling:

- **Element Selector:** Targets all elements of a type (e.g., `p { color: red; }`).
- **Class Selector:** Targets elements with a matching class name, preceded by a dot (e.g., `.container { width: 80%; }`).
- **ID Selector:** Targets a single element with a matching ID, preceded by a hash (e.g., `#header { height: 60px; }`). ID must be unique within the page.
- **Universal Selector:** Targets every element in the DOM using asterisk (e.g., `* { margin: 0; }`).

### 3.3 The CSS Box Model

Every element in HTML is rendered as a rectangular box. The CSS Box Model defines how the size and spacing of this box are calculated.



The dimensions of an element are computed based on the `box-sizing` property:

- **content-box (Default):** Total Width = width + left/right padding + left/right border + left/right margin.
- **border-box:** Total Width = width + left/right margin (padding and border are absorbed inside the specified width).

## Quick Revision — Module 1

Web Development Basics (HTML & CSS) | OE-EC704A

### Core HTML Tags & CSS Syntax

| HTML Form / Tag      | Syntax and Key Attributes                                                                              |
|----------------------|--------------------------------------------------------------------------------------------------------|
| Hyperlink Anchor     | <code>&lt;a href="URL" target="_blank"&gt;Link Text&lt;/a&gt;</code>                                   |
| Embedded Image       | <code>&lt;img src="path" alt="Fallback text" width="500"&gt;</code>                                    |
| HTML Table Structure | <code>&lt;table&gt; &lt;tr&gt; &lt;td colspan="2"&gt;Data&lt;/td&gt; &lt;/tr&gt; &lt;/table&gt;</code> |
| HTML Form Tag        | <code>&lt;form action="/login" method="POST"&gt; inputs... &lt;/form&gt;</code>                        |
| CSS Selector Type    | Syntax and Example Usage                                                                               |
| CSS Syntax Rule      | <code>selector { property: value; }</code>                                                             |
| Class Styling        | <code>.button { background-color: blue; padding: 10px; }</code>                                        |
| ID Styling           | <code>#main-nav { display: flex; justify-content: space-between; }</code>                              |
| Universal Reset      | <code>* { box-sizing: border-box; margin: 0; padding: 0; }</code>                                      |

### One-Line Definitions

- **HTML:** Structured markup language defining content hierarchy of a web document.
- **CSS:** Style sheet language used for presenting and laying out HTML elements.
- **DOM (Document Object Model):** An API representing an HTML document as a logical tree structure.
- **Tag:** A code marker representing the start or end of an HTML element (e.g., `<p>`).
- **Element:** A complete component consisting of start tag, content, and end tag.
- **Attribute:** Name-value pairs inside HTML tags that modify their behavior or supply metadata.
- **Viewport:** The visible area of a web page on a client device screen.
- **URL (Uniform Resource Locator):** The global reference address of a web resource on the Internet.

### Frequently Asked / Viva Questions

#### Q1. What is the difference between GET and POST methods in forms?

GET appends form data directly to the URL as query parameters, rendering it visible and bookmarkable. It is intended for idempotent requests. POST sends data inside the HTTP message body, making it secure for sensitive parameters like passwords.

#### Q2. Why is DOCTYPE html declared at the beginning of an HTML file?

**Frequently Asked / Viva Questions (contd.)**

It is a standard preamble instructing the client browser to render the document in standards-compliant mode rather than quirks mode.

**Q3. What is the difference between `span` and `div` tags?**

`div` is a block-level container element that starts on a new line. `span` is an inline element used to style specific words or content inline without inserting page breaks.

**Q4. How does CSS specificity calculate which style to apply?**

Specificity is calculated as a numeric weight: inline styles hold the highest priority (1000), followed by ID selectors (100), class/attribute selectors (10), and element selectors (1).

**Q5. What is the difference between relative and absolute positioning in CSS?**

Relative positioning offsets an element relative to its normal position in the document flow. Absolute positioning moves the element relative to its closest parent with non-static positioning.

**Q6. What does the `box-sizing: border-box` property do?**

It alters the CSS Box Model calculation so that borders and paddings are included in the element's declared width and height, preventing layout breakage.

**Q7. What are `colspan` and `rowspan` attributes in HTML tables?**

`colspan` merges cells horizontally across columns. `rowspan` merges cells vertically across rows.

**Q8. What is semantic HTML and why is it important?**

Semantic HTML uses tags that explain their purpose clearly (e.g., `<header>`, `<nav>`, `<article>`, `<footer>`) instead of generic `<div>` blocks. It enhances accessibility and SEO search indexing.

**Q9. How do you link CSS files to an HTML document?**

Using the HTML `<link>` element inside the `<head>` block: `<link rel="stylesheet" href="style.css">`.

**Q10. What are CSS pseudoclasses and how are they used?**

Pseudoclasses define special states of elements (e.g., `:hover` for mouse hover, `:active` for clicks, `:focus` for keyboard selection).

**Form Submission: GET vs. POST comparison**

| <b>Metric</b>         | <b>GET Method</b>                               | <b>POST Method</b>                                |
|-----------------------|-------------------------------------------------|---------------------------------------------------|
| <b>Data Location</b>  | Appended to URL (Query parameters).             | Inside HTTP Request Body.                         |
| <b>Security</b>       | Poor (exposed in browser history/logs).         | Higher (hidden from URL path).                    |
| <b>Payload Limits</b> | Limited (typically 2048 characters).            | No architectural payload size limit.              |
| <b>Caching</b>        | Allowed/Cached by proxy servers.                | Never cached.                                     |
| <b>Idempotency</b>    | Idempotent (submitting multiple times is safe). | Non-idempotent (resubmitting duplicates actions). |

# Module 2

## Java Execution Environment

---

### 1. Java Execution Environment

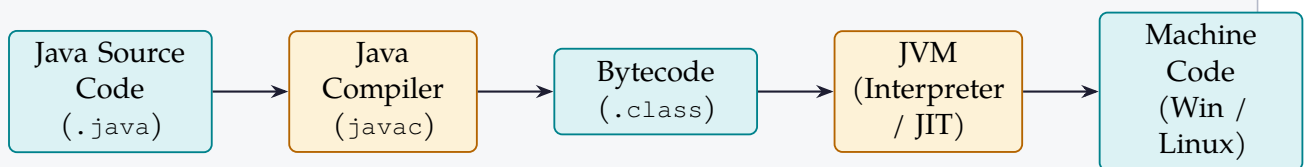
---

Java uses a hybrid execution model combining compilation and interpretation. This ensures its core principle: **Write Once, Run Anywhere (WORA)**.

#### Bytecode and Platform Independence

- **Bytecode:** A highly optimized set of machine instructions designed for execution by the JVM. It is stored in a `.class` file after compilation.
- **Platform Independence:** Java source code compiles into platform-independent bytecode. However, the **JVM itself is platform-dependent** because it must translate bytecode into the host system's native machine instructions.

#### Java Compile-and-Run Execution Flow



#### 1.1 JDK vs. JRE vs. JVM

Understanding the boundary between JVM, JRE, and JDK is crucial:

## JDK, JRE, and JVM Containment Relationship

**JDK (Java Development Kit)**Development Tools: `javac`, `jar`, `jdb`, `javadoc`**JRE (Java Runtime Environment)**Libraries: `rt.jar`, Core API runtime support**JVM (Java Virtual Machine)**

Class Loader, Execution Engine (Interpreter + JIT), GC, Memory Manager

| Metric             | JVM (Virtual Machine)                             | JRE (Runtime)                                                 | JDK (Development)                                                                      |
|--------------------|---------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Composition</b> | Classloader, Execution Engine, Garbage Collector. | JVM + Core class libraries ( <code>rt.jar</code> ) + plugins. | JRE + Compiler ( <code>javac</code> ) + tools ( <code>jar</code> , <code>jdb</code> ). |
| <b>Purpose</b>     | Execution of compiled bytecode.                   | Running Java applications.                                    | Developing and running Java apps.                                                      |
| <b>Target User</b> | None (internal engine).                           | End users/clients.                                            | Developers.                                                                            |

## 2. Java Characteristics and Features

Java's design is guided by a set of primary characteristics (often called "Java Buzzwords"):

### Core Java Characteristics

1. **Simple:** Clean syntax derived from C++ but without complex features like pointers, operator overloading, or multiple inheritance.
2. **Object-Oriented:** Everything in Java is an object (except primitive types), promoting modularity and extensibility.
3. **Robust:** Emphasizes early error checking via compile-time checking, runtime exception handling, and automatic garbage collection.
4. **Secure:** Executes inside a virtual machine sandbox without direct memory manipulation or pointers, preventing buffer overflows and unauthorized system access.
5. **Portable and Architecture-Neutral:** Compiles to bytecode that can run on any device with a compatible JVM, without needing recompilation.
6. **Multithreaded:** Built-in support for concurrent execution of multiple tasks, improving resource utilization in GUI and server applications.

### 3. Object-Oriented Programming (OOP) Concepts

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which can contain data (in the form of fields or attributes) and code (in the form of procedures or methods). Java is a class-based, object-oriented language.

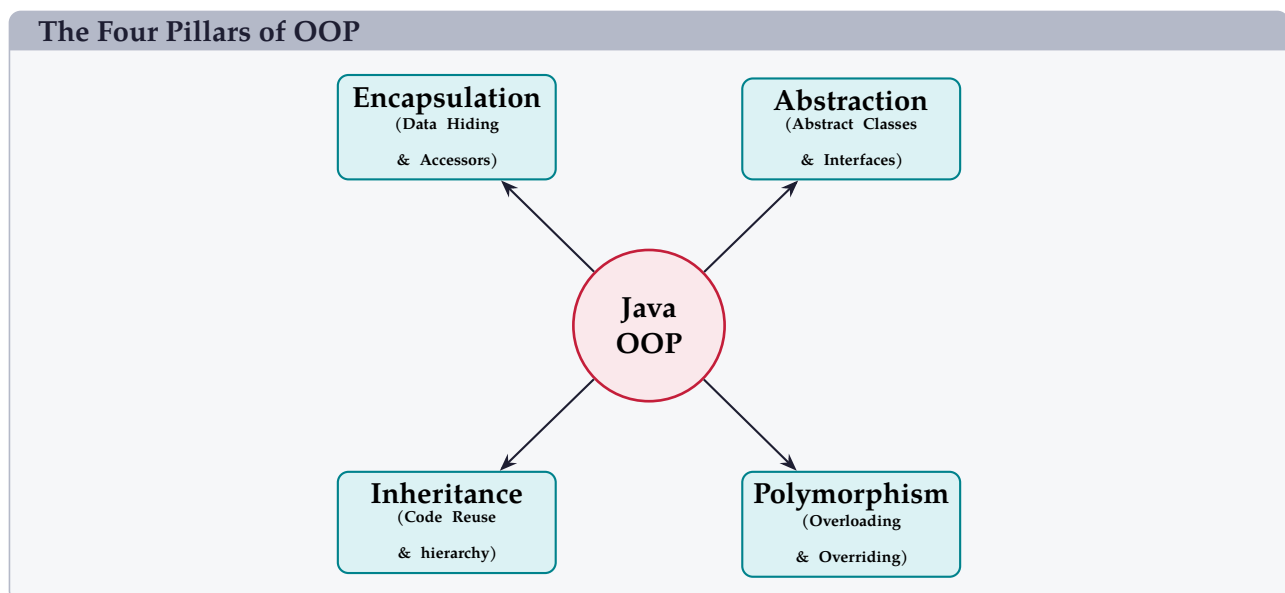
#### Class vs. Object

- **Class:** A logical blueprint or template used to create objects. It defines the state (attributes) and behavior (methods) that all objects of this class will share. It does not occupy memory space.
- **Object:** A physical instance of a class. It has state, behavior, and a unique identity. It occupies physical memory space on the heap.

#### 3.1 The Four Pillars of OOP

Java fully implements the four core principles of object-oriented programming:

1. **Encapsulation (Data Hiding):** The process of wrapping data (variables) and code (methods) together as a single unit (class) while restricting direct access to the state.
  - **Implementation:** Declare instance variables as `private` and provide access via public getter and setter methods.
  - **Advantage:** Provides security, data validation, and flexibility to make variables read-only or write-only.
2. **Abstraction (Implementation Hiding):** Hiding internal implementation details and exposing only the essential features of an object.
  - **Implementation:** Achieved via `abstract` classes and `interfaces`.
  - **Advantage:** Reduces complexity and isolates the user from implementation changes.
3. **Inheritance (Code Reusability):** The mechanism by which one class (subclass) inherits the fields and methods of another class (superclass).
  - **Implementation:** Achieved using the `extends` keyword. Java supports single, multilevel, and hierarchical inheritance but does not support multiple inheritance with classes to avoid ambiguity (the "Diamond Problem").
  - **Advantage:** Promotes code reusability and establishes an "IS-A" relationship.
4. **Polymorphism (Many Forms):** The ability of a single action, method, or object to behave differently depending on the context.
  - **Compile-time (Static) Polymorphism:** Resolved during compilation. Achieved via **Method Overloading** (same method name, different parameters).
  - **Runtime (Dynamic) Polymorphism:** Resolved during runtime. Achieved via **Method Overriding** (same method name and signature in subclass) and **Dynamic Method Dispatch**.



## 4. Structure of a Simple Java Program

Every standard standalone Java application has an entry point class containing a main method.

### 4.1 Standard Hello World Program

Below is a simple Java program that prints a message to the console:

```
public class HelloWorld {
    // Execution entry point
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

### 4.2 Key Elements Explained

- **public class HelloWorld:** Declares a public class named `HelloWorld`. In Java, the file name must match the name of the public class it contains (e.g., `HelloWorld.java`).
- **public static void main(String[] args):** The entry point of the program:
  - **public:** Accessible by the JVM from outside the class.
  - **static:** Can be called without creating an instance of the class.
  - **void:** Returns no value to the runtime environment.
  - **String[] args:** Array of command-line arguments passed as strings.
- **System.out.println():** Built-in output stream to print text followed by a new line to the console.

### 4.3 Compiling and Running Java Programs

Java programs require a two-step compile-and-run sequence:

1. **Compilation:** Run the compiler (`javac`) to convert the source code into bytecode class files:

```
javac HelloWorld.java
```

This generates a file named `HelloWorld.class` containing platform-independent bytecode.

2. **Execution:** Run the JVM interpreter (`java`) to load and execute the class file:

```
java HelloWorld
```

(Note: Do not include the `.class` extension when executing the program).

## 5. Data Types, Operators, and Expressions

---

### 5.1 Primitive vs. Reference Data Types

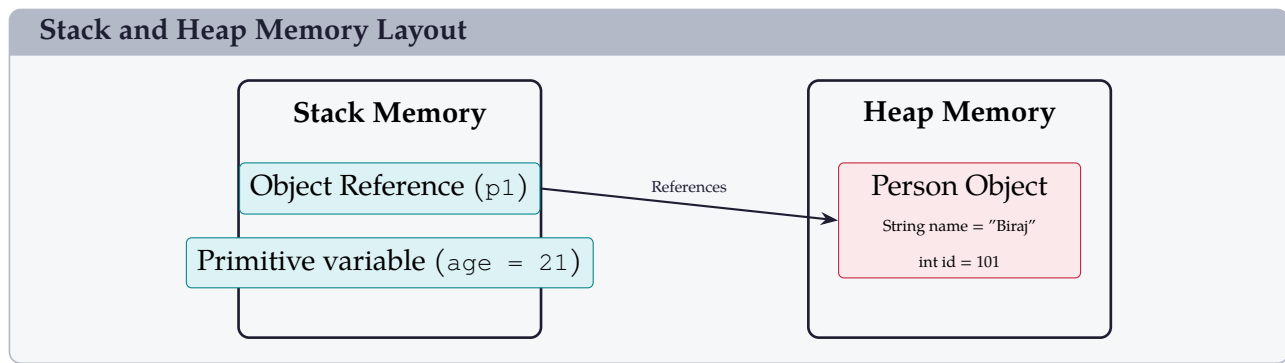
Java enforces static type checking and groups types into two primary categories:

| Feature       | Primitive Data Types                                 | Reference Data Types                        |
|---------------|------------------------------------------------------|---------------------------------------------|
| Allocation    | Stored directly on the stack memory.                 | Object on Heap; address reference on Stack. |
| Default Value | 0, 0.0, or false (never null).                       | null by default.                            |
| Size          | Fixed sizing (e.g., int: 32-bit, double: 64-bit).    | Depends on the host system architecture.    |
| Examples      | byte, short, int, long, float, double, char, boolean | Classes, interfaces, arrays, strings.       |

### 5.2 Memory Allocation Model

Java allocates memory across two distinct spaces during runtime:

- **Stack Memory:** Stores primitive variables and local references. Uses LIFO (Last In First Out) allocation.
- **Heap Memory:** Dynamic memory pool that hosts all runtime objects and instance variables.



### 5.3 Operators and Expressions

Java provides a comprehensive set of operators to build computational expressions:

- **Arithmetic Operators:** Binary (+, -, \*, /, %) and Unary (++, --).
- **Relational Operators:** Compare operands for equality or order (==, !=, <, >, <=, >=).
- **Logical Operators:** Combine boolean expressions (&& (logical AND), || (logical OR), ! (logical NOT)). Supports short-circuit evaluation.
- **Bitwise Operators:** Perform bit-level manipulations (&, |, ^, ~, <<, >>, >>>).
- **Ternary Operator:** Conditional assignment shorthand (variable = (condition) ? valueIfTrue : valueIfFalse).

## 6. Control Flow and Branching

Java supports selection, iteration, and jump statements to control execution paths:

- **Selection Statements:** `if-else` blocks for boolean conditions and `switch` blocks for multi-way branching (supports byte, short, char, int, Enums, and String).
- **Iteration Statements:**
  - `for` loops for index-based loops.
  - `while` loops for condition-controlled loops (pre-test).
  - `do-while` loops for exit-controlled loops (post-test, runs at least once).
- **Jump Statements:**
  - `break`: Terminated loop or switch block immediately.
  - `continue`: Skips the rest of the current loop body and proceeds to the next iteration step.
  - **Labeled Statements:** Loops can be labeled (e.g., `outerLoop: for(...)`) to specify which loop a nested `break` or `continue` targets.

## Quick Revision — Module 2

Introduction to Java Programming & OOP | OE-EC704A

### Java Basic Syntax & Loop Structures

| Concept             | Syntax and Example Usage                                                        |
|---------------------|---------------------------------------------------------------------------------|
| Simple Program      | <pre>public class Test { public static void main(String[] args) { ... } }</pre> |
| Data Type Sizing    | byte (8-bit), char (16-bit Unicode), int (32-bit), double (64-bit)              |
| Ternary Operator    | <pre>int min = (a &lt; b) ? a : b;</pre>                                        |
| Short-Circuit logic | <pre>if (obj != null &amp;&amp; obj.isValid()) { ... } // prevents NPE</pre>    |
| Loop Statements     | Conditional and Iterative Formats                                               |
| for Loop            | <pre>for(int i = 0; i &lt; n; i++) { ... }</pre>                                |
| while Loop          | <pre>while(condition) { statements; }</pre>                                     |
| do-while Loop       | <pre>do { statements; } while(condition);</pre>                                 |
| Labeled Break       | <pre>outer: for(...) { if(cond) break outer; }</pre>                            |

### One-Line Definitions

- **JVM (Java Virtual Machine):** The abstract execution engine that interprets and executes compiled Java bytecode.
- **Bytecode:** Platform-independent intermediate code representation generated by `javac`.
- **JDK:** Java Development Kit containing compiler, tools, and the JRE runtime.
- **JRE:** Java Runtime Environment providing JVM and core system class libraries.
- **Stack Memory:** LIFO memory space storing local variables and function execution frames.
- **Heap Memory:** General dynamic memory pool where all Java objects are instantiated.
- **Short-Circuiting:** Logical expression evaluation that stops as soon as the outcome is guaranteed.
- **Platform Independence:** The ability of bytecode to run unmodified on any OS running a JVM.
- **Encapsulation:** Wrapping variables and methods into a class and restricting direct access.
- **Abstraction:** Hiding execution details and exposing only the interface contract.
- **Inheritance:** Class relationship mechanism enabling child reuse of parent methods/variables.

- **Polymorphism:** Feature allowing a single entity to assume different behaviors at runtime or compile-time.

### Frequently Asked / Viva Questions

#### Q1. Why is Java platform-independent but JVM is platform-dependent?

Java bytecode is standardized and can run on any machine. However, the JVM must be custom-tailored to the target operating system to translate that bytecode to native CPU instructions.

#### Q2. What is JIT Compiler in Java?

The Just-In-Time (JIT) compiler is a component of the JVM execution engine. It compiles frequently executed bytecode (hot spots) directly into native machine code at runtime to boost performance.

#### Q3. What is the difference between `path` and `classpath` variables?

`path` is an OS environment variable that locates shell commands like `javac` and `java`. `classpath` is a JVM-specific setting directing the classloader where to find compiled user class files.

#### Q4. What are the JVM memory partitions?

JVM memory is divided into: Method Area (class metadata, static variables), Heap (objects), Stack (local variables, execution frames), PC Register (current instruction address), and Native Method Stack.

#### Q5. Why does Java use Unicode for characters?

Java was designed for global applications. While ASCII uses 8 bits and represents 256 characters, Unicode uses 16 bits, allowing it to represent characters from all major languages.

#### Q6. What is the difference between prefix and postfix increment operators?

Prefix (`++i`) increments the variable first and then returns the incremented value. Postfix (`i++`) returns the original value first and then increments the variable.

#### Q7. Explain short-circuit logical operators.

In `A && B`, if `A` is false, the JVM skips evaluating `B` since the result is guaranteed to be false. Similarly, in `A || B`, if `A` is true, `B` is not evaluated.

#### Q8. Can we use a String in a switch-case statement?

Yes, starting from Java 7, String literals are supported in switch-case statements. The JVM compares strings using their `equals()` values and hash codes.

#### Q9. What is the difference between `break` and `continue` statements?

`break` terminates the loop completely, passing control to the statement after the loop. `continue` terminates only the current loop iteration, jumping to the loop update/condition.

## Frequently Asked / Viva Questions (contd.)

**Q10. Why is there no `goto` statement in Java?**

Java does not support `goto` to enforce clean structured programming principles and prevent "spaghetti code." Instead, labeled `break` and `continue` statements are provided.

**Q11. Explain the four pillars of OOP and their implementation in Java.**

The four pillars are: Abstraction (hiding implementation details via abstract classes and interfaces), Encapsulation (data hiding using private attributes and public getters/setters), Inheritance (code reuse using subclassing with the `extends` keyword), and Polymorphism (methods behaving differently based on context via method overloading and overriding).

**Q12. Why is the main method in Java declared as `public static void`?**

`public` makes it accessible to the JVM from outside the class context. `static` allows the JVM to execute the method without instantiating the class first. `void` specifies that it returns no value to the operating system after execution completes.

## Java Iteration Statement Comparisons

| Feature         | <code>for</code> Loop          | <code>while</code> Loop         | <code>do-while</code> Loop |
|-----------------|--------------------------------|---------------------------------|----------------------------|
| Loop Structure  | Counter-based iteration.       | Condition-controlled iteration. | Exit-controlled iteration. |
| Condition Check | Evaluated at start.            | Evaluated at start.             | Evaluated at end.          |
| Min. Runs       | 0 runs if condition fails.     | 0 runs if condition fails.      | Guaranteed at least 1 run. |
| Best Use Case   | Iterating when count is known. | Loop until event triggers.      | Interactive menus.         |

# Module 3

## Class Fundamentals and Objects

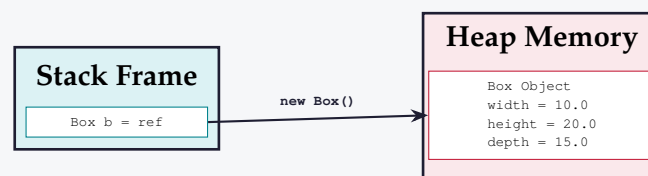
### 1. Class Fundamentals and Objects

In Java, all program logic resides within classes. A class acts as a blueprint, and objects are active physical instances of that blueprint.

#### Class Declaration & Object Creation

- **Declaring a Class:** Declared using the `class` keyword. It encapsulates state (instance variables) and behavior (methods).
- **Creating Instances:** Objects are created dynamically on the heap using the `new` keyword, which allocates memory and invokes the constructor.

#### Object Creation and Reference Linkage



#### 1.1 Constructors in Java

A constructor initializes the state of a newly created object. It has no return type and must have the same name as the class.

- **Default Constructor:** Automatically provided by the compiler if no explicit constructor is defined. It initializes primitives to their default values (e.g., 0, false) and object references to `null`.
- **Parameterized Constructor:** User-defined constructor that accepts parameters to initialize instances with custom values.
- **Copy Constructor:** A constructor that initializes a new object using an existing object of the same class:

```
class Box {
    double width, height;
    Box(Box ob) { // Copy Constructor
```

```
        this.width = ob.width;
        this.height = ob.height;
    }
}
```

## 1.2 Argument Passing in Java

Java uses a single parameter passing mechanism: **Pass-by-Value**.

- **Primitive Arguments:** A copy of the value is passed. Changes made inside the method do not affect the original variable.
- **Object Arguments:** A copy of the reference address is passed. The method can modify the object's contents (since they both point to the same object on the heap), but reassigning the reference variable inside the method does not affect the caller's original reference.

## 1.3 The `static` Keyword

The `static` keyword is used to create class-level members that are shared across all instances.

- **Static Variables:** Single memory copy allocated in the method area, shared by all objects of that class.
- **Static Methods:** Methods that can be called without creating an instance. They can only access static data and call other static methods. They cannot use the `this` or `super` keywords.
- **Static Blocks:** Used to initialize static variables. Executed automatically when the class is first loaded into memory, before the main method or object constructors.

## 1.4 Nested and Inner Classes

Java allows defining a class inside another class.

- **Member Inner Class:** A non-static class declared inside another. It has access to all members (including private ones) of the outer class. It requires an instance of the outer class to exist.
- **Static Nested Class:** Declared static. It behaves like a top-level class packaged inside another class. It cannot access non-static outer members.
- **Local Inner Class:** Defined inside a block or method. It can only access local variables if they are effectively final.
- **Anonymous Inner Class:** An inner class defined inline without a name, typically used for overriding listener methods or implementing interfaces on the fly.

# 2. Inheritance and Reuse Models

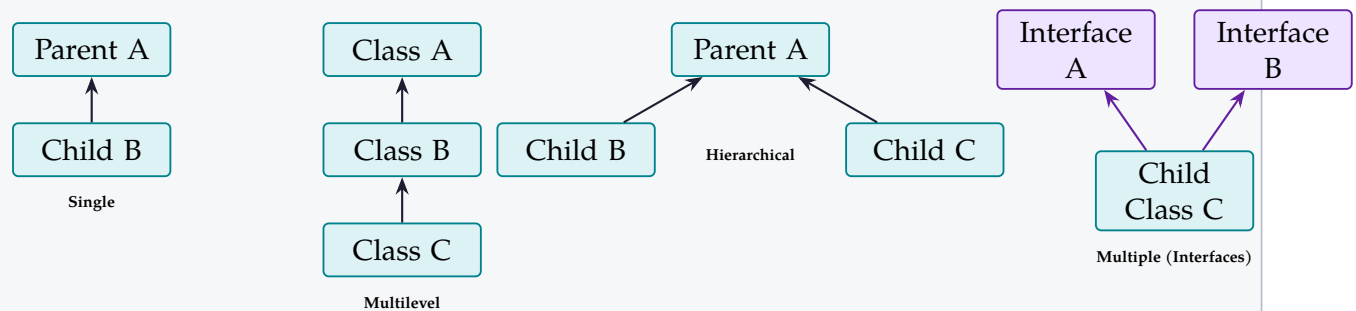
---

Inheritance enables hierarchical classifications. In Java, classes are extended using the `extends` keyword.

### Inheritance Characteristics

- **Single Inheritance Only:** Java classes can only extend **one** parent class to prevent the diamond problem (ambiguity under duplicate variable/method signatures).
- **The `super` Keyword:** Used to invoke the parent class constructor (must be the first statement in the subclass constructor), access overridden parent methods, or read hidden parent variables.

### Inheritance Trees supported in Java



## 2.1 Composition vs. Inheritance

Design patterns advocate favoring object composition over class inheritance:

| Metric                | Composition (Has-A Relationship)                 | Inheritance (Is-A Relationship)                |
|-----------------------|--------------------------------------------------|------------------------------------------------|
| <b>Coupling Level</b> | Loose coupling; internal details remain private. | Tight coupling; breaks encapsulation boundary. |
| <b>Flexibility</b>    | Behaviors can change dynamically at runtime.     | Static; behavior fixed at compilation time.    |
| <b>Design Reuse</b>   | Reuses functional components as class members.   | Reuses code by expanding a class hierarchy.    |
| <b>Example</b>        | Car has an Engine instance.                      | Car is a subclass of Vehicle.                  |

## 3. Polymorphism and Runtime Resolution

### 3.1 Method Overloading vs. Overriding

These represent static and dynamic polymorphism:

| Feature          | Method Overloading (Static)                | Method Overriding (Dynamic)                    |
|------------------|--------------------------------------------|------------------------------------------------|
| Class Context    | Occurs within the same class definition.   | Occurs across child-parent subclass hierarchy. |
| Method Signature | Name must be same; parameters must differ. | Signature (parameters, returns) must match.    |
| Resolution Time  | Compile-time (static binding).             | Runtime (dynamic binding).                     |
| Keywords         | None required.                             | Assisted by <code>@Override</code> annotation. |

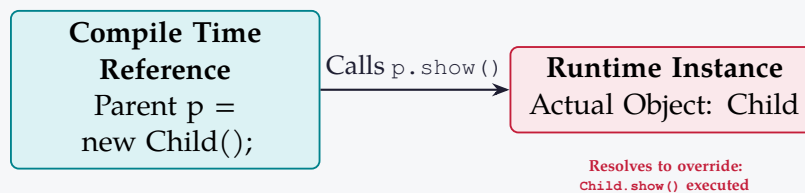
### 3.2 Dynamic Method Dispatch

Dynamic Method Dispatch is the mechanism by which a call to an overridden method is resolved at runtime rather than compile-time.

#### Dynamic Method Dispatch Rule

When an overridden method is called through a superclass reference, Java determines which version of the method to execute based on the **type of the actual object** being referred to at runtime, not the type of the reference variable.

#### Dynamic Method Dispatch Flow



### 3.3 Abstract Classes and the `final` Keyword

- **Abstract Classes:** Declared using the `abstract` keyword. They cannot be instantiated. They can contain abstract methods (signatures without implementations) and concrete methods. They act as partial templates for subclasses.
- **The `final` Keyword:** Used to restrict inheritance and modifications:
  - `final` Variable: Value cannot be reassigned (behaves as a constant).
  - `final` Method: Cannot be overridden by subclasses.
  - `final` Class: Cannot be inherited/extended.

### 3.4 Access Modifiers

Java regulates member visibility using four access levels:

- **`private`:** Visible only within the declaring class.
- **Default (no modifier):** Visible only to classes inside the same package.
- **`protected`:** Visible to the same package and subclass child files in external packages.
- **`public`:** Accessible from any class anywhere in the project.

## Quick Revision — Module 3

Classes, Inheritance & Polymorphism | OE-EC704A

### Class, Constructor & Inheritance Syntax

| Concept                  | Syntax and Example Usage                                                  |
|--------------------------|---------------------------------------------------------------------------|
| Subclass Extension       | <code>class Child extends Parent { ... }</code>                           |
| Super Constructor Call   | <code>Child(int val) { super(val); ... }</code>                           |
| Copy Constructor         | <code>Box(Box b) { this.width = b.width; this.height = b.height; }</code> |
| Static Block             | <code>static { staticVar = 10; }</code>                                   |
| Polymorphism & Variables | Core Declarations & Reference Types                                       |
| Dynamic Dispatch Ref     | <code>Parent p = new Child(); p.overriddenMethod();</code>                |
| Abstract Class           | <code>abstract class Shape { abstract void draw(); }</code>               |
| Final Method             | <code>final void display() { ... } // cannot be overridden</code>         |
| Final Variable           | <code>final int MAX_SPEED = 120; // cannot be reassigned</code>           |

### One-Line Definitions

- **Class:** A user-defined template that defines the structure and behaviors of objects.
- **Object:** A physical instance of a class allocated in memory.
- **Copy Constructor:** A constructor that initializes a new object using data from an existing object of the same class.
- **Pass-by-Value:** Parameter passing where a copy of the argument value (or reference address) is passed.
- **Static Method:** A method bound to the class itself, executable without class instantiation.
- **Dynamic Method Dispatch:** Dynamic binding resolution resolving overridden method calls at runtime.
- **Abstract Class:** A class that cannot be instantiated and may contain abstract methods.
- **Inner Class:** A class declared inside the body of another enclosing class.

### Frequently Asked / Viva Questions

#### Q1. Does Java support pass-by-reference?

No, Java is strictly pass-by-value. When passing primitive variables, their values are copied. When passing object references, a copy of the heap memory address is passed.

**Frequently Asked / Viva Questions (contd.)****Q2. Can an abstract class have constructors? Why?**

Yes. Although an abstract class cannot be instantiated directly, its constructor is called during subclass instantiation via `super()` to initialize its fields.

**Q3. What is the difference between `this()` and `super()` constructor calls?**

`this()` invokes another constructor within the same class. `super()` invokes a constructor of the parent superclass. Both must be the first line of their respective constructors.

**Q4. What is the difference between method overloading and overriding?**

Overloading occurs in the same class and is resolved at compile-time (different parameters). Overriding occurs in subclasses and is resolved at runtime (same method signature).

**Q5. Why can't static methods use the `this` keyword?**

The `this` keyword refers to the current object instance. Static methods run without any object instance context, so there is no `this` instance to reference.

**Q6. What is a copy constructor and why do we need it?**

A copy constructor takes an object of the same class as a parameter to copy its fields into a new object. It provides control over whether a shallow or deep copy of the object is created.

**Q7. What are the types of inner classes in Java?**

Java inner classes are: member inner class (non-static, declared inside class), static nested class (declared static inside class), local inner class (declared inside method), and anonymous inner class (unnamed class defined inline).

**Q8. How does Dynamic Method Dispatch work?**

When an overridden method is called through a parent class reference variable, Java resolves it at runtime based on the actual object instance type being referenced, not the reference type.

**Q9. What is the difference between a shallow copy and a deep copy?**

A shallow copy copies the object's reference fields (sharing nested objects). A deep copy creates new instances of nested objects, making the new copy independent.

**Q10. Can a final method be overloaded?**

Yes. A final method cannot be overridden (redefined in a subclass), but it can be overloaded (defined with different parameters within the same class).

## Java Access Modifier Visibility Matrix

| Modifier         | Same Class | Same Package | Subclass | Everywhere |
|------------------|------------|--------------|----------|------------|
| <b>private</b>   | Yes        | No           | No       | No         |
| <b>Default</b>   | Yes        | Yes          | No       | No         |
| <b>protected</b> | Yes        | Yes          | Yes      | No         |
| <b>public</b>    | Yes        | Yes          | Yes      | Yes        |

# Module 4

## Interfaces

---

### 1. Interfaces

---

An interface is a reference type in Java that is similar to a class but contains only constants, method signatures, default methods, static methods, and nested types.

#### Interface Concept

An **Interface** defines a contract that implementing classes must satisfy. It specifies *what* a class should do, but not *how* it does it.

- Declared using the `interface` keyword.
- All variables declared inside an interface are implicitly `public`, `static`, and `final` (constants).
- All methods without a body are implicitly `public` and `abstract`.

#### 1.1 Syntax and Implementation

A class implements an interface using the `implements` keyword. It must provide concrete overrides for all abstract methods declared in the interface.

```
interface Flyable {
    int SPEED_LIMIT = 300; // Implicitly public static final
    void fly();             // Implicitly public abstract
}

class Airplane implements Flyable {
    @Override
    public void fly() { // Must be declared public
        System.out.println("Airplane is flying at high altitude.");
    }
}
```

#### 1.2 Interface Additions in Java 8 and 9

To support backward compatibility and clean API design, Java has expanded what an interface can contain:

- **Default Methods (Java 8):** Declared using the `default` keyword. They provide a default concrete implementation, allowing developers to add new methods to interfaces without breaking existing implementing classes.
- **Static Methods (Java 8):** Concrete methods bound to the interface itself. They cannot be overridden by implementing classes.
- **Private Methods (Java 9):** Used to encapsulate common helper logic shared among default/static methods within the same interface.

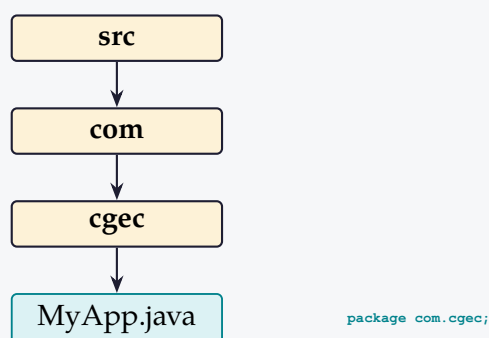
## 2. Packages

A package is a grouping of related classes, interfaces, and subpackages, providing namespace management and access protection.

### Java Packages

- **Directory Mapping:** Packages map directly to the folder structure on disk. For example, a class in package `com.cgec.webtech` must reside in a folder path `com/cgec/webtech/`.
- **Namespace Clashing Prevention:** Ensures class name uniqueness by grouping them under a domain namespace.
- **The `package` Keyword:** Declared as the very first non-comment statement in a source file.
- **The `import` Keyword:** Used to make classes from other packages accessible within the current file.

### Java Package Structure Mapping to File System



## 3. Access Control Mechanism

Java uses access modifiers to control visibility of classes, constructors, variables, and methods.

- **`private`:** Visible only within the declaring class. It provides the highest encapsulation.

- **Default (Package-Private):** Visible to all classes in the same package. No keyword is specified.
- **protected:** Visible to all classes in the same package, and to subclasses in different packages.
- **public:** Accessible from any class anywhere in the application.

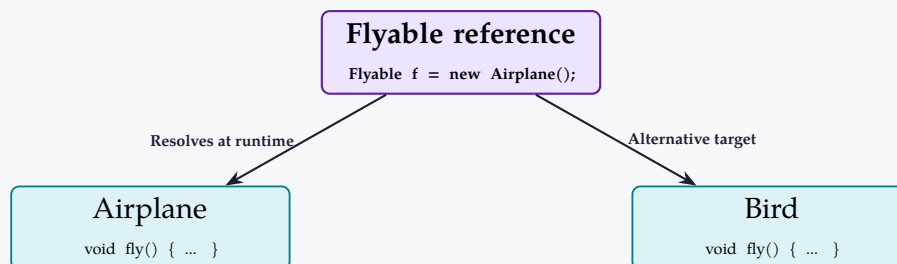
## 4. Dynamic Method Lookup

Dynamic Method Lookup (or Interface-based Dynamic Binding) is the runtime mechanism by which the JVM resolves calls to interface methods.

### Interface Callbacks

When an interface reference calls a method, the JVM looks up the actual runtime class of the referenced object, retrieves its dynamic method table, and invokes the concrete overridden method. This is the foundation of loose coupling and polymorphic callbacks.

### Interface Callback Mechanism



## Quick Revision — Module 4

Interfaces and Packages | OE-EC704A

### Interface & Package Declarations

| Concept              | Syntax and Example Usage                                               |
|----------------------|------------------------------------------------------------------------|
| Interface Definition | <code>interface InterfaceName { void methodSignature(); }</code>       |
| Implementation       | <code>class ClassName implements InterfaceName { ... }</code>          |
| Default Method       | <code>default void log() { System.out.println("Default Log"); }</code> |
| Static Method        | <code>static int add(int a, int b) { return a + b; }</code>            |
| Packages & Imports   | Core syntax headers                                                    |
| Package Declaration  | <code>package com.cgec.webtech; // must be first line</code>           |
| Class Import         | <code>import java.util.ArrayList;</code>                               |
| Wildcard Import      | <code>import java.util.*;</code>                                       |
| Static Import        | <code>import static java.lang.Math.PI;</code>                          |

### One-Line Definitions

- **Interface:** A reference type declaring method signatures and constants, defining a contract.
- **Package:** A directory structure grouping related classes and interfaces.
- **Default Method:** An interface method containing a default body, declared with the `default` keyword.
- **Access Modifier:** Keywords controlling the accessibility level of classes and class members.
- **Dynamic Method Lookup:** Runtime resolution of interface method calls based on actual object instance type.
- **Encapsulation:** Bundling class state and behavior while restricting direct access using `private`.
- **Namespace:** A naming container resolving name conflicts across different components.
- **Multiple Inheritance:** Implementing multiple interfaces to inherit behaviors from multiple sources.

### Frequently Asked / Viva Questions

**Q1. Can an interface be instantiated? Why?**

No. An interface does not have constructors and contains abstract method

**Frequently Asked / Viva Questions (contd.)**

signatures without implementations, so it cannot represent concrete runtime state.

**Q2. Why does Java allow multiple inheritance using interfaces but not classes?**

Classes hold state (instance variables). If a class inherits from multiple parent classes, duplicate variable names cause conflicts. Since interfaces hold no state variables (only constants) and historically had no implementations, there is no duplicate implementation ambiguity.

**Q3. What is the difference between an Interface and an Abstract Class?**

An abstract class can contain instance variables (state) and constructors, whereas an interface can only contain public static final constants. A class can extend only one class but implement multiple interfaces.

**Q4. What are default methods and why were they introduced in Java 8?**

Default methods have concrete bodies declared with the `default` keyword inside an interface. They allow adding new methods to interfaces without breaking existing classes that implement the interface.

**Q5. What is a functional interface?**

An interface containing exactly one abstract method. It can be annotated with `@FunctionalInterface` and is used for Lambda expressions (e.g., `Runnable`, `Comparable`).

**Q6. Can we declare an interface method as `private` or `protected`?**

Interface abstract methods must be `public`. However, starting from Java 9, we can declare `private` and `private static` helper methods inside interfaces. `protected` is never allowed.

**Q7. What is the default access modifier in an interface?**

All fields are implicitly `public static final`. All abstract methods are implicitly `public abstract`.

**Q8. What is a static import in Java?**

A static import allows importing static members (variables and methods) of a class directly (e.g., `import static java.lang.Math.sqrt;`). This allows invoking them without prepending the class name.

**Q9. What is the difference between `package` and `import` statements?**

The `package` statement declares which package/directory the current class belongs to. The `import` statement tells the compiler where to find other classes referenced in the file.

**Q10. How does the JVM resolve name conflicts between classes in different imported packages?**

If a class with the same name exists in two imported packages (e.g., `java.util.Date` and `java.sql.Date`), the compiler forces the developer to use the fully qualified class name in code to resolve ambiguity.

## Java Access Modifier Visibility Matrix

| Modifier         | Same Class | Same Package | Subclass | Everywhere |
|------------------|------------|--------------|----------|------------|
| <b>private</b>   | Yes        | No           | No       | No         |
| <b>Default</b>   | Yes        | Yes          | No       | No         |
| <b>protected</b> | Yes        | Yes          | Yes      | No         |
| <b>public</b>    | Yes        | Yes          | Yes      | Yes        |

# Module 5

## Java Exception Handling Mechanism

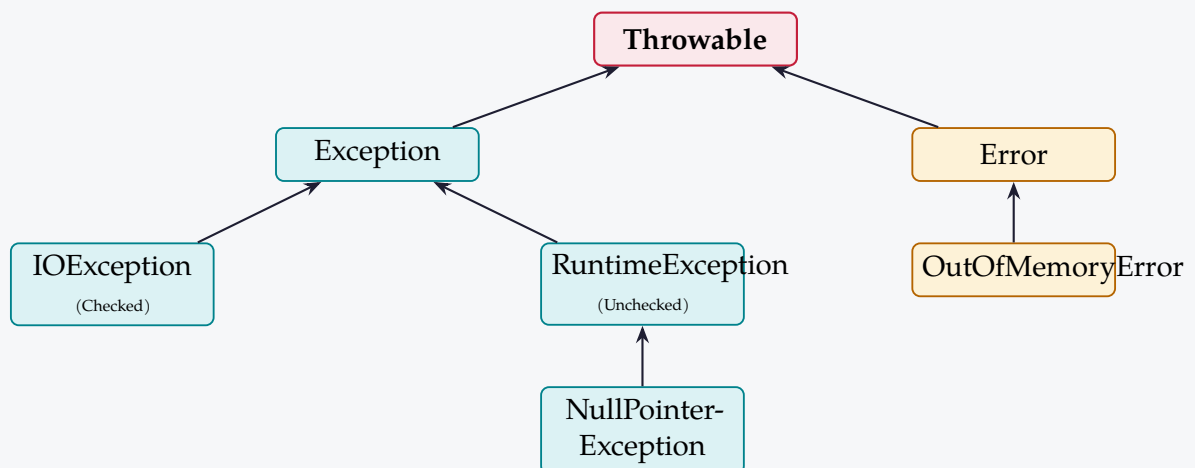
### 1. Java Exception Handling Mechanism

An exception is an unwanted or unexpected event that occurs during the execution of a program (at runtime) and disrupts the normal flow of instructions.

#### Exception Handling Goals

The primary goal of exception handling is to prevent runtime program crashes and allow the application to handle errors gracefully, releasing resources and reporting descriptive messages.

#### Java Exception Class Hierarchy



#### 1.1 Throwable Hierarchy

The root class of the exception hierarchy is `java.lang.Throwable`. Only objects that are instances of this class (or subclasses) can be thrown by the JVM or the Java `throw` statement.

- **Error:** Extends `Throwable`. Represents serious problems that a reasonable application should not try to catch (e.g., `VirtualMachineError`, `OutOfMemoryError`, `StackOverflowError`). They represent infrastructure or JVM failures.
- **Exception:** Extends `Throwable`. Represents conditions that a reasonable application might want to catch and handle.

## 1.2 Checked vs. Unchecked Exceptions

- **Checked Exceptions:** Classes that extend `Exception` but do not extend `RuntimeException`. They are verified by the compiler at compile-time. The program must handle them using try-catch or declare them using the `throws` clause (e.g., `IOException`, `SQLException`, `ClassNotFoundException`).
- **Unchecked Exceptions:** Classes that extend `RuntimeException`. They are not checked by the compiler. They usually result from logical errors or bad program state (e.g., `NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`).

## 2. Exception Keywords & Handling Structures

---

Java exception handling is managed via five keywords: `try`, `catch`, `finally`, `throw`, and `throws`.

### 2.1 Try-Catch-Finally Blocks

- **try:** Surrounds the block of code that might throw an exception. It must be followed by at least one `catch` block or a `finally` block.
- **catch:** Defines the exception handler. Multiple catch blocks can follow a single try block, starting from the most specific exception subclass to the most general (superclass).
- **finally:** Executed guaranteed, whether an exception occurs or not, and even if there is a return statement inside try/catch. Used for cleanup tasks like closing resources.

### 2.2 Throw vs. Throws

- **throw:** Used to explicitly throw an instance of any class extending `Throwable` from code (e.g., `throw new ArithmeticException("Divide by zero");`).
- **throws:** Used in method declarations to list the checked exceptions that the method might throw, delegating the responsibility of handling to the caller method (e.g., `public void read() throws IOException { ... }`).

### 2.3 Try-with-Resources and the `AutoCloseable` Interface

Introduced in Java 7, the try-with-resources statement is a try block declared with one or more resource declarations.

```
try (BufferedReader br = new BufferedReader(new FileReader("test.txt"))) {  
    System.out.println(br.readLine());  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

- Any class declared inside the try parentheses must implement the `java.lang.AutoCloseable` interface.
- The JVM automatically closes these resources when the try block exits, eliminating the need for verbose manual resource-closing logic inside a `finally` block.

## 2.4 User-Defined / Custom Exceptions

Developers can create custom exceptions to represent domain-specific errors.

- To create a checked exception, extend the `java.lang.Exception` class.
- To create an unchecked exception, extend the `java.lang.RuntimeException` class.

```
class InvalidAgeException extends Exception {  
    public InvalidAgeException(String message) {  
        super(message);  
    }  
}
```

## Quick Revision — Module 5

Exception Handling | OE-EC704A

### Exception Handling Code Constructs

| Structure            | Syntax and Example Usage                                                                           |
|----------------------|----------------------------------------------------------------------------------------------------|
| Try-Catch Block      | <pre>try { int val = 10 / 0; } catch(ArithmeticException e) { ... }</pre>                          |
| Try-With-Resources   | <pre>try (FileReader fr = new FileReader("file.txt")) { ... }</pre>                                |
| Method Throws Clause | <pre>public void load() throws IOException, ClassNotFoundException { ... }</pre>                   |
| Explicit Throw       | <pre>throw new NullPointerException("Null Reference Encountered");</pre>                           |
| Custom Exception     | <pre>class CustomException extends RuntimeException { CustomException(String m){super(m);} }</pre> |

### One-Line Definitions

- **Exception:** An event disrupting the normal instruction execution flow.
- **Checked Exception:** An exception subclass checked at compile-time, requiring try-catch or throws.
- **Unchecked Exception:** An exception subclass extending `RuntimeException`, not checked at compile-time.
- **Error:** A fatal system condition that applications should not attempt to handle.
- **Try Block:** A block enclosing code that might generate a runtime exception.
- **Finally Block:** A block executing guaranteed cleanup actions, running after try-catch completion.
- **Try-with-Resources:** An automatic resource management block closing `AutoCloseable` targets automatically.
- **AutoCloseable:** Interface defining the `close()` contract for automatic try-with-resources cleanups.

### Frequently Asked / Viva Questions

#### Q1. What is the difference between Checked and Unchecked exceptions?

Checked exceptions are verified by the compiler at compile-time (e.g., `IOException`) and must be handled or declared. Unchecked exceptions are `RuntimeExceptions` (e.g., `NullPointerException`) and are not compiler-checked.

#### Q2. What is the difference between `final`, `finally`, and `finalize`?

**Frequently Asked / Viva Questions (contd.)**

`final` is a keyword making variables unassignable, methods override-protected, and classes inheritance-protected. `finally` is a block for resource cleanup in exception handling. `finalize` is a method invoked by the Garbage Collector before object reclamation.

**Q3. Can we have a try block without a catch block?**

Yes. A try block can be followed by a `finally` block instead of a catch block, or it can be a try-with-resources statement which handles closing automatically.

**Q4. What is Exception Propagation?**

If an exception is not caught in the method where it occurred, it propagates down the call stack to the calling method. This repeats until it is handled or the JVM terminates the thread.

**Q5. What is class `Throwable`? What are its main subclasses?**

`Throwable` is the root class of all exceptions and errors in Java. Its main subclasses are `java.lang.Exception` and `java.lang.Error`.

**Q6. How does multi-catch work in Java 7?**

A single catch block can handle multiple unrelated exceptions using the pipe symbol (e.g., `catch (IOException | SQLException e)`). The variable `e` is implicitly `final`.

**Q7. What happens if an exception is thrown in a `finally` block?**

If an exception is thrown in a `finally` block, it overrides and suppresses any active exception previously thrown in the try or catch blocks, unless handled internally.

**Q8. Why should more specific catch blocks be declared before general ones?**

Java matches catch blocks from top to bottom. If a general exception catch block (like `Exception`) is placed first, it will catch all subclass exceptions, making subsequent specific catch blocks unreachable and causing compile errors.

**Q9. What is class `AutoCloseable`?**

It is a functional interface introduced in Java 7 containing a single method: `void close() throws Exception`. Classes implementing this interface are automatically closed in a try-with-resources construct.

**Q10. Is it possible to throw a checked exception from static initializer block?**

No. A static initializer block runs when the class is loaded and cannot throw checked exceptions since there is no calling method to catch or declare them.

## Checked vs. Unchecked Exception Comparison

| Metric            | Checked Exception                                   | Unchecked Exception                               |
|-------------------|-----------------------------------------------------|---------------------------------------------------|
| Compilation Check | Checked at compile-time by compiler.                | Ignored by compiler during build.                 |
| Handling Rule     | Must use try-catch or declare throws.               | Optional to handle; fix logic instead.            |
| Base Class        | Directly extends <code>java.lang.Exception</code> . | Extends <code>java.lang.RuntimeException</code> . |
| Root Cause        | External environment issues (I/O, network).         | Program logical errors (null checks, math).       |

# Module 6

## String Basics and Immutability

### 1. String Basics and Immutability

In Java, strings are represented by the `String` class inside the `java.lang` package.

#### String Immutability Concept

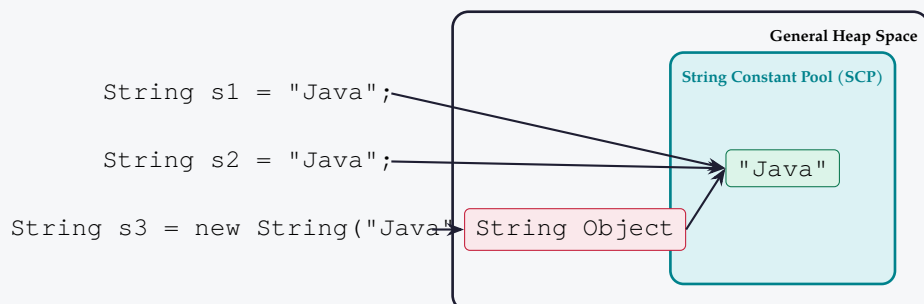
Java `String` objects are **immutable**, meaning their values cannot be changed once they are created in memory. Any modification operation (like concatenation or case conversion) does not alter the original string; instead, it returns a reference to a completely new `String` object.

#### 1.1 String Constant Pool (SCP)

To save memory and increase performance, JVM maintains a special memory area inside the heap called the **String Constant Pool (SCP)**.

- **Literal Allocation:** When a string is created using a literal (e.g., `String s1 = "Java";`), JVM first checks the SCP. If the string is already present, it returns the existing reference. Otherwise, it allocates a new string in the pool.
- **New Object Allocation:** Using the `new` keyword (e.g., `String s2 = new String("Java");`) bypasses pool checking and creates a new object in the general heap space, which references the literal value in the SCP.

#### String Memory References: General Heap vs. String Constant Pool (SCP)



### 2. String vs. StringBuffer

When a program performs frequent modifications to character sequences, using the immutable `String` class generates high amounts of garbage references. Java provides `StringBuffer` to resolve this.

| Metric            | String Class                        | StringBuffer Class                      |
|-------------------|-------------------------------------|-----------------------------------------|
| Mutability        | Immutable (read-only).              | Mutable (can modify in-place).          |
| Performance       | Slower for modifications.           | Fast and memory efficient for loops.    |
| Thread Safety     | Naturally thread-safe.              | Thread-safe (methods are synchronized). |
| Memory Allocation | Allocations inside the SCP or Heap. | Allocated strictly in the general Heap. |

### 3. Constructors and Core Operations

#### 3.1 String Constructors

- `String()`: Initializes a newly created, empty `String` object.
- `String(char[] chars)`: Allocates a new `String` from an array of characters.
- `String(String original)`: Creates a copy of the argument string.

#### 3.2 Core String Operations

| Operation  | Method Signature                                  | Functional Description                                       |
|------------|---------------------------------------------------|--------------------------------------------------------------|
| Extraction | <code>char charAt(int index)</code>               | Returns the character at the specified index.                |
|            | <code>int length()</code>                         | Returns the character count.                                 |
| Comparison | <code>boolean equals(Object obj)</code>           | Value-based comparison for character equality.               |
|            | <code>int compareTo(String another)</code>        | Lexicographically compares two strings (returns difference). |
| Searching  | <code>int indexOf(String str)</code>              | Returns the index of the first occurrence.                   |
|            | <code>boolean contains(CharSequence s)</code>     | Returns true if the string matches the pattern.              |
| Modifying  | <code>String substring(int start, int end)</code> | Returns characters from start (incl.) to end (excl.).        |
|            | <code>String trim()</code>                        | Removes leading and trailing whitespace.                     |

### 4. Utility Conversions

Java provides two major methods to perform conversions to strings:

- **toString() Method:** Defined in the root `Object` class. Every Java class inherits it and can override it to provide a custom string representation of the object's fields.
- **String.valueOf() Static Method:** Overloaded methods to convert primitive data types (like `int`, `double`, `boolean`) or objects into string representations. If an object is passed, it returns `obj.toString()` or `"null"` safely if the reference is null.

## 5. StringBuffer Operations

---

`StringBuffer` maintains a growable character buffer, defaulting to 16 characters of extra capacity.

- **append():** Appends data to the end of the buffer.
- **insert():** Inserts data at a specified index.
- **reverse():** Reverses the sequence of characters in-place.
- **Capacity vs. Length:** `length()` returns the actual number of characters stored, while `capacity()` returns the total allocated buffer space.

## Quick Revision — Module 6

String Handling | OE-EC704A

### String & StringBuffer Syntax

| Operation               | Syntax and Example Usage                                               |
|-------------------------|------------------------------------------------------------------------|
| Literal Assignment      | <code>String s1 = "Biraj";</code> // Allocated in SCP                  |
| Object Constructor      | <code>String s2 = new String("Biraj");</code> // Heap object reference |
| Character Extraction    | <code>char c = s1.charAt(2);</code> // 'r'                             |
| Substring Extraction    | <code>String sub = s1.substring(1, 3);</code> // "ir"                  |
| StringBuffer Operations | Mutable buffer syntax                                                  |
| Declaration             | <code>StringBuffer sb = new StringBuffer("CGE");</code>                |
| Append Operation        | <code>sb.append("C");</code> // sb becomes "CGEC"                      |
| Insert Operation        | <code>sb.insert(3, "-");</code> // sb becomes "CGE-C"                  |
| In-place Reverse        | <code>sb.reverse();</code> // sb becomes "C-EGC"                       |

### One-Line Definitions

- **Immutability:** State constraint preventing modification of heap values after initial allocation.
- **String Constant Pool:** A cached region in Java heap memory storing unique string literals.
- **StringBuffer:** A thread-safe, synchronized, mutable character sequence class.
- **StringBuilder:** A non-synchronized, fast, mutable character sequence class.
- **equals():** Overridden method in String checking value-based character equality.
- **== Operator:** Operator checking whether two reference variables point to the same memory address.
- **valueOf():** Static String utility method converting primitives and objects to strings.
- **toString():** Standard Object method returning class representation details as a String.

### Frequently Asked / Viva Questions

#### Q1. Why are String objects immutable in Java?

To enable String Constant Pool caching (saving heap memory), ensure thread-safety, and prevent security vulnerabilities (since connection strings and file paths cannot be altered).

#### Q2. What is the difference between `equals()` and `==` when comparing Strings?

**Frequently Asked / Viva Questions (contd.)**

`==` compares reference equality (whether variables point to the exact same object reference). The `equals()` method checks value equality (whether character sequences are identical).

**Q3. How many objects are created in the statement: `String s = new String("Hello");`?**

Two objects. One literal object is placed in the String Constant Pool (if not already present), and a second String object is created in the general heap space.

**Q4. What does the method `intern()` do?**

It returns the canonical representation of the string from the String Constant Pool. If the pool already contains the string, it returns its reference; otherwise, it adds the string to the pool and returns the reference.

**Q5. What is the difference between `StringBuffer` and `StringBuilder`?**

Both are mutable classes. `StringBuffer` is thread-safe and has synchronized methods, causing high performance overhead. `StringBuilder` is not synchronized, making it faster and preferred for single-threaded applications.

**Q6. What is the default capacity of a `StringBuffer`? How does it increase?**

The default initial capacity is 16. When the buffer capacity is exceeded, it grows automatically using the formula:  $\text{New Capacity} = (\text{Old Capacity} \times 2) + 2$ .

**Q7. Can we override the static method `valueOf()` in custom classes?**

No. Static methods belong to the class scope and are resolved statically during compilation. They cannot be overridden in subclasses. Custom classes override `toString()` instead.

**Q8. What does the method `compareTo()` return?**

It returns 0 if strings are equal, a negative integer if the current string is lexicographically smaller than the argument, and a positive integer if it is larger.

**Q9. Why does `System.out.println(obj)` invoke the `toString()` method?**

The print streams implicitly check if the argument is an object. If yes, it calls `String.valueOf(obj)`, which internally invokes `obj.toString()` to get a printable string.

**Q10. What is the purpose of the `trim()` method in the `String` class?**

It returns a copy of the string with all leading and trailing whitespaces (characters with Unicode code point less than or equal to `'U+0020'`) removed.

**Comparison: String vs. StringBuffer vs. StringBuilder**

| <b>Metric</b>        | <b>String</b>              | <b>StringBuffer</b>         | <b>StringBuilder</b>   |
|----------------------|----------------------------|-----------------------------|------------------------|
| <b>Mutability</b>    | Immutable.                 | Mutable.                    | Mutable.               |
| <b>Thread Safety</b> | Yes (due to immutability). | Yes (synchronized methods). | No (non-synchronized). |
| <b>Performance</b>   | Slowest for edits.         | Slower (lock overhead).     | Fastest.               |
| <b>Introduced in</b> | Java 1.0                   | Java 1.0                    | Java 1.5               |

# Module 7

## Java I/O Stream Basics

### 1. Java I/O Stream Basics

An Input/Output (I/O) Stream represents an abstraction of a data source (e.g., keyboard, file) or destination (e.g., console, network socket).

#### Stream Concept

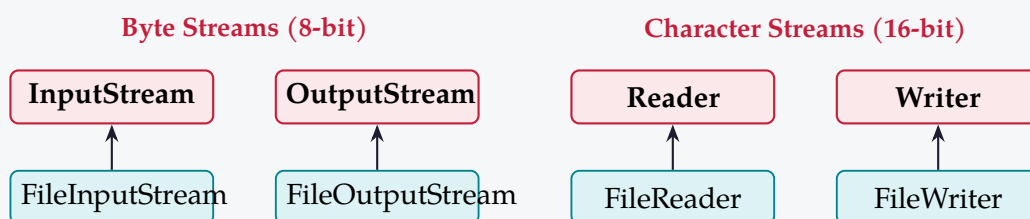
A **Stream** is a continuous sequence of data bytes. Input streams are used to read data from a source, and Output streams are used to write data to a target.

#### 1.1 Byte Stream vs. Character Stream

Java's I/O library operates on two primary categories of streams:

- **Byte Streams:** Process data as raw 8-bit bytes. They are suitable for binary data (images, sound, zip files).
- **Character Streams:** Process data as 16-bit Unicode characters. They handle translation of text encoding models automatically.

#### Java Stream Hierarchy



### 2. Console I/O Operations

Java represents console streams using three static variables declared in the `System` class:

- **`System.in`:** Standard Input Stream (type `InputStream`), mapped to keyboard input.
- **`System.out`:** Standard Output Stream (type `PrintStream`), mapped to console output.
- **`System.err`:** Standard Error Stream (type `PrintStream`), mapped to console error output.

## 2.1 Reading Console via BufferedReader

Because `System.in` is a byte stream, we wrap it in a character stream (`InputStreamReader`) and pass it to a `BufferedReader` to enable line-by-line reading:

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
System.out.print("Enter your name: ");
String name = br.readLine();
```

## 2.2 Reading Console via Scanner

The `java.util.Scanner` class parses primitives and strings using regular expressions:

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter an integer: ");
int value = sc.nextInt();
```

# 3. File I/O Operations

---

## 3.1 FileInputStream and FileOutputStream

Used to read and write raw bytes from/to a file on disk:

```
// Copying a binary file byte-by-byte
try (FileInputStream fis = new FileInputStream("input.jpg");
     FileOutputStream fos = new FileOutputStream("output.jpg")) {
    int byteData;
    while ((byteData = fis.read()) != -1) {
        fos.write(byteData);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

## 3.2 FileReader and FileWriter

Used to read and write character text data from/to files on disk:

```
try (FileReader fr = new FileReader("file.txt");
     FileWriter fw = new FileWriter("copy.txt")) {
    int charData;
    while ((charData = fr.read()) != -1) {
        fw.write(charData);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

## 3.3 Buffered Stream Wrapper Pattern

Disk I/O requests are expensive. To improve performance, we wrap base stream classes inside buffered wrapper classes:

- **BufferedReader:** Wraps around a `Reader` (e.g., `FileReader`) to buffer characters, providing a highly efficient `readLine()` method.
- **BufferedWriter:** Wraps around a `Writer` to buffer written characters, writing to disk only when the buffer is full or when `flush()` is invoked.

## Quick Revision — Module 7

Java I/O Streams | OE-EC704A

### Java I/O Stream Syntax

| Construct                           | Syntax and Example Usage                                                             |
|-------------------------------------|--------------------------------------------------------------------------------------|
| Console <code>BufferedReader</code> | <pre>BufferedReader br = new BufferedReader(new InputStreamReader(System.in));</pre> |
| Console <code>Scanner</code>        | <pre>Scanner sc = new Scanner(System.in); int val = sc.nextInt();</pre>              |
| File Reader (Text)                  | <pre>FileReader fr = new FileReader("input.txt"); int c = fr.read();</pre>           |
| File Writer (Text)                  | <pre>FileWriter fw = new FileWriter("output.txt"); fw.write("Hello");</pre>          |
| File Byte Input                     | <pre>FileInputStream fis = new FileInputStream("in.dat"); fis.read();</pre>          |
| Buffered Writer Wrapper             | <pre>BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt"));</pre>        |

### One-Line Definitions

- **Stream:** A logical abstraction of a continuous sequence of data bytes.
- **Byte Stream:** Stream processing raw 8-bit binary data units.
- **Character Stream:** Stream processing 16-bit Unicode character data units.
- **`InputStreamReader`:** A bridge class converting byte streams into character streams.
- **`BufferedReader`:** A character stream wrapper that caches characters in memory to reduce disk calls.
- **`Scanner`:** A text scanner utility class parsing primitives using token delimiters.
- **`System.in`:** Static instance of `InputStream` mapped to the client keyboard.
- **`System.out`:** Static instance of `PrintStream` mapped to the standard console output.

### Frequently Asked / Viva Questions

#### Q1. What is the difference between Byte Streams and Character Streams?

Byte streams read/write data as 8-bit bytes (base classes: `InputStream`, `OutputStream`), ideal for binary files. Character streams read/write data as 16-bit Unicode characters (base classes: `Reader`, `Writer`), ideal for text files.

#### Q2. Why do we wrap a `FileReader` inside a `BufferedReader`?

Reading directly from disk is slow. Wrapping in a `BufferedReader` caches character chunks in memory, reducing physical disk reads, and exposes the convenient

**Frequently Asked / Viva Questions (contd.)**

`readLine()` method.

**Q3. What is the purpose of the `InputStreamReader` class?**

It acts as a bridge, converting a byte stream (like `System.in`) into a character stream, translating bytes into characters using the default system charset.

**Q4. What is the difference between `PrintStream` and `PrintWriter`?**

`PrintStream` is a byte stream class handling 8-bit output (like `System.out`).  
`PrintWriter` is a character stream class handling 16-bit Unicode character output.

**Q5. What does the method `flush()` do in output streams?**

It forces the output stream to write any cached buffered bytes/characters immediately to the destination device (like the disk file), emptying the local memory buffer.

**Q6. What is the default buffer size of a `BufferedReader`?**

The default buffer size is 8192 characters (8 KB), which is large enough to handle most typical text file read sizes efficiently.

**Q7. How does the `Scanner` class handle delimiters?**

By default, the `Scanner` class uses whitespace (spaces, tabs, newlines) as the token delimiter. This can be customized using the `useDelimiter()` method.

**Q8. What is the difference between `read()` method of `InputStream` and `Reader`?**

`InputStream.read()` reads a single byte and returns it as an integer (0 to 255).  
`Reader.read()` reads a single Unicode character and returns it as an integer (0 to 65535). Both return `-1` when reaching EOF.

**Q9. What checked exception is commonly thrown during file operations?**

`java.io.IOException` and its subclass `java.io.FileNotFoundException`. They must be explicitly caught or declared using a `throws` clause.

**Q10. What are the standard console stream fields inside the `System` class?**

`System.in` (standard input - `InputStream`), `System.out` (standard output - `PrintStream`), and `System.err` (standard error - `PrintStream`).

**Comparison: Reader/Writer vs. InputStream/OutputStream**

| <b>Metric</b>            | <b>Reader / Writer Hierarchy</b>       | <b>InputStream / OutputStream Hierarchy</b> |
|--------------------------|----------------------------------------|---------------------------------------------|
| <b>Data Unit</b>         | 16-bit Unicode character unit.         | 8-bit raw byte unit.                        |
| <b>Primary Target</b>    | Text documents and strings.            | Images, sound, ZIP, raw binary data.        |
| <b>Root Packages</b>     | Lives inside <code>java.io</code>      | Lives inside <code>java.io</code>           |
| <b>Default EOF value</b> | Returns <code>-1</code> at stream end. | Returns <code>-1</code> at stream end.      |

# Module 8

## Java Collections Framework Overview

---

### 1. Java Collections Framework Overview

---

The Collections Framework, declared inside the `java.util` package, provides a unified architecture to store and manipulate a group of objects.

#### Core Design Benefits

- Reduces programming effort by providing standard data structures.
- Boosts speed and quality via high-performance, optimized implementations.
- Promotes software reuse by offering polymorphic operations.

#### 1.1 Core Interfaces

The framework is structured hierarchically around a set of core interfaces:

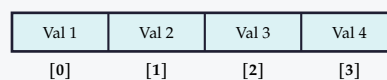
- **Collection:** The root interface defining basic operations (size, add, remove, clear).
- **List:** Extends `Collection`. Represents an ordered sequence of elements that can contain duplicates. Supports index-based access.
- **Set:** Extends `Collection`. A collection that cannot contain duplicate elements, modeling mathematical sets.
- **Queue:** Extends `Collection`. Designed to hold elements prior to processing (typically FIFO).

### 2. ArrayList vs. LinkedList

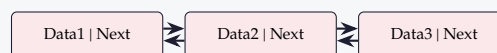
| Metric             | ArrayList                           | LinkedList                                   |
|--------------------|-------------------------------------|----------------------------------------------|
| Internal Structure | Dynamically resizing array.         | Doubly linked list nodes.                    |
| Access Time        | $O(1)$ random access (index-based). | $O(n)$ search access (sequential traverse).  |
| Insertion/Deletion | $O(n)$ (requires element shifts).   | $O(1)$ if pointer is located.                |
| Memory Overhead    | Low (only stores elements).         | Higher (stores element and prev/next links). |

#### ArrayList vs. LinkedList Memory Layout

##### ArrayList (Contiguous Memory):



##### LinkedList (Pointer Links):



## 3. Accessing and Iterating Collections

Java provides two primary mechanisms to traverse through collection elements:

### 3.1 The Iterator Interface

An iterator provides sequential access to collection elements. It is retrieved using `iterator()` method on any Collection object.

```

ArrayList<String> list = new ArrayList<>();
list.add("Java");
list.add("HTML");

Iterator<String> itr = list.iterator();
while (itr.hasNext()) {
    String element = itr.next();
    System.out.println(element);
}

```

- **Methods:** `hasNext()` (returns true if more elements exist), `next()` (returns next element), and `remove()` (removes current element safely during iteration).
- **Fail-Fast Behavior:** Iterators on standard collections like `ArrayList` throw a `ConcurrentModificationException` if the collection is structurally modified (add/remove) outside the iterator's own methods during iteration.

### 3.2 The For-Each Statement

Introduced in Java 5, the enhanced `for` loop (for-each) provides a cleaner syntax to iterate over arrays and collections:

```
for (String element : list) {  
    System.out.println(element);  
}
```

Internally, the compiler translates this syntax into standard Iterator-based code for objects implementing the `java.lang.Iterable` interface.

## Quick Revision — Module 8

Java Utility package - Collections | OE-EC704A

### Collection Syntax

| Construct              | Syntax and Example Usage                                                                    |
|------------------------|---------------------------------------------------------------------------------------------|
| ArrayList Declaration  | <code>ArrayList&lt;String&gt; list = new ArrayList&lt;String&gt;();</code>                  |
| ArrayList Insert       | <code>list.add("Java"); list.add(1, "CSS"); // inserts at index 1</code>                    |
| LinkedList Declaration | <code>LinkedList&lt;Integer&gt; lList = new LinkedList&lt;&gt;();</code>                    |
| Iterator Traversal     | <code>Iterator&lt;String&gt; itr = list.iterator();<br/>while(itr.hasNext()) { ... }</code> |
| For-Each Loop          | <code>for (String str : list) { System.out.println(str); }</code>                           |

### One-Line Definitions

- **Collections Framework:** Architecture of interfaces and classes for grouping multiple objects.
- **ArrayList:** Resizable-array implementation of the List interface, backing elements contiguously.
- **LinkedList:** Doubly-linked node implementation of the List and Deque interfaces.
- **Iterator:** Object traversal cursor enabling forward traversal and safe element removals.
- **Fail-Fast Iterator:** Iterator throwing exception immediately if collection structure changes during loop.
- **Iterable:** Interface indicating target object can be traversed using enhanced for-each.
- **Generics:** Compiler feature ensuring type-safety in collections (e.g., `ArrayList<String>`).
- **Auto-boxing:** Implicit compiler conversion between primitives and their wrapper classes.

### Frequently Asked / Viva Questions

#### Q1. What is the difference between `ArrayList` and `Vector`?

`ArrayList` is non-synchronized, fast, and not thread-safe. `Vector` is synchronized, has thread-safe methods, and carries significant execution overhead.

#### Q2. Why does a `LinkedList` consume more memory than an `ArrayList`?

An `ArrayList` stores elements contiguously inside a backing array. A `LinkedList` wraps each element inside a node object that also holds reference addresses for the previous and next nodes.

**Frequently Asked / Viva Questions (contd.)****Q3. What is fail-fast behavior in collections?**

It is a JVM runtime validation that throws a `ConcurrentModificationException` if a collection is modified structurally (elements added or deleted) while an active iterator is traversing it.

**Q4. How does `ArrayList` grow its internal array capacity?**

When an element is added and the array is full, `ArrayList` allocates a new array and copies elements. Starting from Java 10, it increases capacity by roughly 50% ( $\text{New Capacity} = \text{Old Capacity} + (\text{Old Capacity} \gg 1)$ ).

**Q5. What is the purpose of the `Iterable` interface?**

It is the superinterface of the `Collection` interface. Implementing `Iterable` requires the class to return an `Iterator`, allowing instances to be used in the enhanced for-each loop.

**Q6. Can we store primitive data types (like `int`, `char`) inside collections?**

No, collections only store object references. However, Java handles this automatically using **Autoboxing**, which converts primitives to their wrapper objects (e.g., `int` to `Integer`) when inserted.

**Q7. What is the difference between `Iterator` and `ListIterator`?**

`Iterator` can traverse any `Collection` in a forward direction. `ListIterator` can only traverse `Lists`, but supports bidirectional traversal (forward and backward) and element modifications.

**Q8. Which `List` implementation is preferred for frequent search operations?**

`ArrayList` is preferred because it supports  $O(1)$  constant time lookup via index-based access, whereas `LinkedList` requires  $O(n)$  linear traversal.

**Q9. Which `List` implementation is preferred for frequent insertions at the beginning of the list?**

`LinkedList` is preferred. Inserting at the beginning requires only updating two node pointers ( $O(1)$ ), whereas `ArrayList` requires shifting all existing elements, taking  $O(n)$  time.

**Q10. What is the difference between `List` and `Set` interfaces?**

A `List` maintains insertion order and can contain duplicate elements. A `Set` is unordered and cannot contain duplicate elements.

## ArrayList vs. LinkedList operations complexity

| Operation             | ArrayList                | LinkedList                      |
|-----------------------|--------------------------|---------------------------------|
| Get by Index          | $O(1)$ constant time.    | $O(n)$ linear traversal.        |
| Insert/Delete (Start) | $O(n)$ shift overhead.   | $O(1)$ pointer change.          |
| Insert/Delete (End)   | $O(1)$ amortized.        | $O(1)$ tail reference update.   |
| Memory consumption    | Low (consecutive slots). | High (two references per node). |

# Module 9

## Introduction to Applets

---

### 1. Introduction to Applets

---

An applet is a specialized Java program designed to be embedded within an HTML web page and executed on a client machine inside a web browser sandbox.

#### Applet Security Sandbox

To prevent malicious code execution on client machines, applets run under strict security constraints:

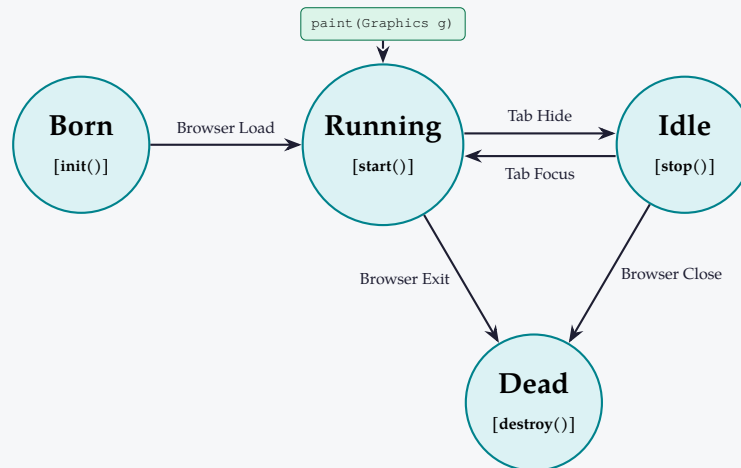
- Applets cannot read or write files on the local client disk.
- Applets cannot communicate with any host except the one they were downloaded from.
- Applets cannot execute local system commands or load native runtime libraries.

#### 1.1 Applet Lifecycle & Execution

An applet inherits from `java.applet.Applet` and does not contain a `main()` method. Instead, its execution is controlled by the browser or the Java Applet Viewer tool through five lifecycle callback methods:

- **`init()`**: Invoked once when the applet is first loaded to perform initialization (e.g., setting up GUI components, reading parameters).
- **`start()`**: Invoked after `init()` or whenever the user returns to the web page containing the applet. Used to start or resume execution threads.
- **`paint(Graphics g)`**: Invoked when the applet window needs to be redrawn (e.g., first display, resizing, or revealing). It uses the `java.awt.Graphics` object.
- **`stop()`**: Invoked when the user navigates away from the page, pausing background threads to save CPU cycles.
- **`destroy()`**: Invoked once before unloading the applet completely to free up system resources.

### Applet Life Cycle State Transitions



## 2. Applet Display and Status Window

Since `Applet` extends the `AWT Panel` container class, it supports all standard component styling methods:

- **`setBackground(Color c) / setForeground(Color c)`**: Configures the background canvas color and text drawing color.
- **`showStatus(String msg)`**: Displays the specified text message in the status bar of the browser or Applet Viewer, providing interactive feedback.

## 3. HTML Embedding and Parameter Passing

Applets are embedded in web pages using the HTML `<applet>` tag.

### 3.1 HTML Applet Tag Syntax

```

<applet code="MyApplet.class" width="300" height="200" codebase="bin/">
  <param name="fontName" value="Courier">
  <param name="fontSize" value="14">
  Alt Text: Applets are not supported by your browser.
</applet>

```

- **`code`**: Specifies the compiled applet class filename.
- **`width & height`**: Declares the size of the applet execution area on the web page.
- **`codebase`**: Specifies the base directory URL containing the class bytecode.

### 3.2 Parameter Retrieval via `getParameter()`

The `<param>` tags allow passing configuration parameters to the applet during load time. Inside the applet's `init()` method, their values are read as Strings:

```
String fontStr = getParameter("fontName"); // Returns "Courier"  
int size = Integer.parseInt(getParameter("fontSize")); // Parses 14
```

### 3.3 getCodeBase() and getDocumentBase()

To locate resources (images, audio clips) dynamically:

- **getCodeBase()**: Returns the base URL of the directory containing the compiled class bytecode.
- **getDocumentBase()**: Returns the URL of the HTML document embedding the applet.

## Quick Revision — Module 9

Java Applets | OE-EC704A

### HTML Applet Tag & Parameters

| Construct       | Syntax and Example Usage                                                               |
|-----------------|----------------------------------------------------------------------------------------|
| HTML Applet Tag | <code>&lt;applet code="MyApp.class" width="400" height="300"&gt;&lt;/applet&gt;</code> |
| Param Value Tag | <code>&lt;param name="bgColor" value="red"&gt;</code>                                  |
| Parameter Query | <code>String val = getParameter("bgColor");</code>                                     |
| Status Output   | <code>showStatus("Applet loading completed successfully");</code>                      |
| Get Code URL    | <code>URL codeBaseUrl = getCodeBase();</code>                                          |
| Get Doc URL     | <code>URL docBaseUrl = getDocumentBase();</code>                                       |

### One-Line Definitions

- **Applet:** A client-side Java program executed inside a browser container sandbox.
- **Sandbox:** A secure runtime environment preventing unauthorized access to client resources.
- **init():** The first applet lifecycle method called once to initialize variables and UI.
- **paint():** AWT component method redrawing text and graphics on the applet canvas.
- **getParameter():** Applet method retrieving parameter values defined in HTML.
- **getCodeBase():** Applet method returning the directory URL containing the class file.
- **getDocumentBase():** Applet method returning the URL of the hosting HTML document.
- **AppletViewer:** A JDK utility tool designed to test and execute Java applets without a browser.

### Frequently Asked / Viva Questions

#### Q1. Which class does every Java Applet extend? Which package contains it?

Every Applet extends `java.applet.Applet` (or `javax.swing.JApplet` for Swing). The class resides inside the `java.applet` package.

#### Q2. Why does an applet not contain a `main()` method?

Applets do not run as standalone applications. They are executed by a web browser container, which handles loading and calls their lifecycle methods.

#### Q3. What is the difference between `init()` and `start()` methods?

`init()` is called once when the applet is first loaded to handle initialization.

**Frequently Asked / Viva Questions (contd.)**

`start()` is called every time the user visits or returns to the page.

**Q4. What is the purpose of the `paint(Graphics g)` method? Who calls it?**

The `paint()` method renders the applet graphics (text, lines, shapes) on the screen. It is called by the browser JVM when the applet first displays or needs redrawing.

**Q5. How do you pass parameters from HTML to an Applet?**

Using the `<param>` tag inside the `<applet>` tag in HTML, and querying it in the applet using the `getParameter(String name)` method.

**Q6. What is the difference between `getCodeBase()` and `getDocumentBase()`?**

`getCodeBase()` returns the URL of the directory containing the applet's compiled class files. `getDocumentBase()` returns the URL of the HTML document embedding the applet.

**Q7. What is the security sandbox model in applets?**

The sandbox restricts applets to prevent access to the client filesystem, executing external processes, or making network connections to hosts other than the source server.

**Q8. Can an applet load native libraries using `System.loadLibrary()`?**

No. Under the default security policy sandbox, applets are prohibited from loading native dynamic libraries to prevent unsafe pointer manipulations on the host.

**Q9. What is the difference between local applets and remote applets?**

Local applets are stored and executed on the same local computer without network access. Remote applets are downloaded from a remote web server over the network and executed locally.

**Q10. Why are Java Applets deprecated in modern web development?**

They require browser plugins (NPAPI) that pose major security risks. Modern browsers have disabled plugin support in favor of HTML5, CSS3, and JavaScript standards.

**Applet Lifecycle Methods Transition Matrix**

| Method            | Invocation Frequency                    | Primary Purpose                                                   |
|-------------------|-----------------------------------------|-------------------------------------------------------------------|
| <b>init ()</b>    | Invoked once when first loaded.         | Initializes variables, loads fonts, images, and GUI elements.     |
| <b>start ()</b>   | Invoked on load and page returns.       | Resumes applet execution, starts animations or threads.           |
| <b>paint ()</b>   | Invoked on display and redraw requests. | Renders text, shapes, and images to the screen coordinate system. |
| <b>stop ()</b>    | Invoked on page exit or tab switch.     | Suspends threads, pauses animations to save CPU resources.        |
| <b>destroy ()</b> | Invoked once before unloading.          | Reclaims resources, kills threads, and cleans up memory.          |

# Module 10

## Delegation Event Model

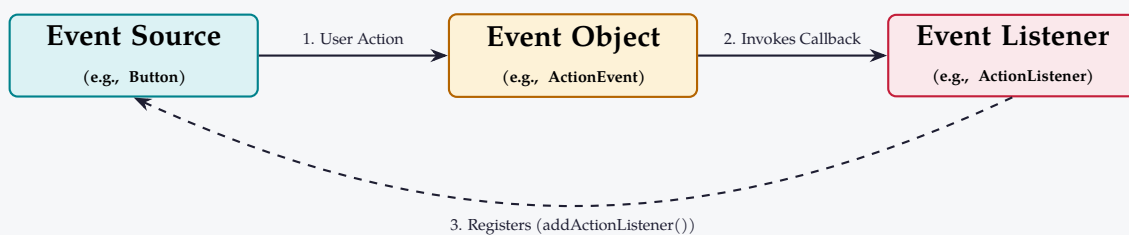
### 1. Delegation Event Model

Java event handling is built on the **Delegation Event Model**, which separates user interface elements from the logical event processing code.

#### Core Model Components

- **Event Source:** The GUI component that generates the event when acted upon (e.g., `Button`, `TextField`).
- **Event Object:** An object encapsulating details of the state change (e.g., `ActionEvent`, `MouseEvent`).
- **Event Listener:** An interface defining callback methods to receive and process specific events.

#### Delegation Event Model Architecture



#### 1.1 Common Event Classes & Listeners

| Event Source                       | Event Class        | Listener Interface                                            |
|------------------------------------|--------------------|---------------------------------------------------------------|
| <b>Button,</b><br><b>TextField</b> | <b>ActionEvent</b> | <code>ActionListener</code>                                   |
| <b>Component</b>                   | <b>MouseEvent</b>  | <code>MouseListener</code> , <code>MouseMotionListener</code> |
| <b>Component</b>                   | <b>KeyEvent</b>    | <code>KeyListener</code>                                      |
| <b>Frame, Window</b>               | <b>WindowEvent</b> | <code>WindowListener</code>                                   |

### 2. Adapter Classes

An event listener interface may declare multiple abstract methods. For example, `WindowListener` has seven methods. Implementing the interface directly forces the developer to write empty method bodies for all seven.

#### Adapter Class Pattern

An **Adapter Class** is a convenience class that implements a listener interface with empty body definitions. By extending the adapter class (e.g., extending `WindowAdapter`), you only need to override the specific methods you require.

### 3. Inner and Anonymous Classes

To register listeners cleanly without polluting the outer class namespace, Java developers utilize nested class patterns:

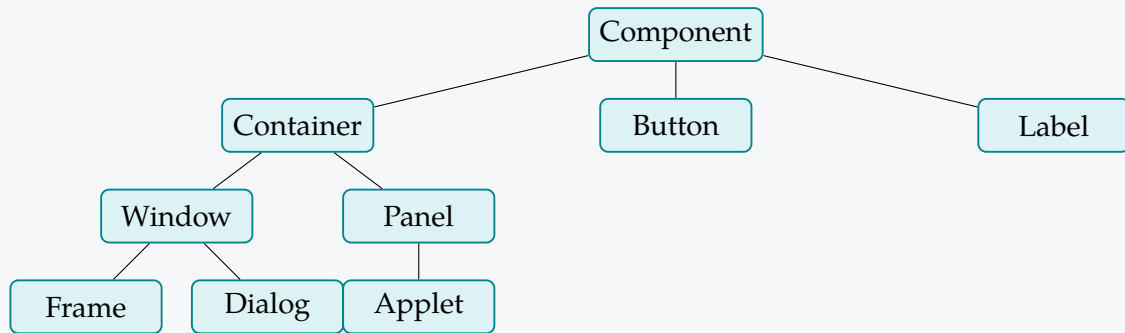
- **Inner Classes:** Define a private class inside the AWT frame that implements the listener, accessing the frame's components directly.
- **Anonymous Inner Classes:** Define an inline implementation of the listener/adaptor class without naming it:

```
btn.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        System.out.println("Button Clicked!");  
    }  
});
```

### 4. AWT Controls

The Abstract Window Toolkit (AWT) provides platform-dependent heavyweight user interface controls:

- **Label:** Displays a single line of read-only text.
- **Button:** A clickable control that generates action events.
- **TextField / TextArea:** Single-line and multi-line text input fields.
- **Checkbox / Choice:** Selection controls representing boolean states and drop-down lists.

**AWT Component Class Hierarchy**

## 5. Layout Managers

Layout managers coordinate the sizing and spatial arrangement of components inside a container:

- **FlowLayout:** Default for `Panel` and `Applet`. Places components in a single line, wrapping left-to-right, top-to-bottom.
- **BorderLayout:** Default for `Frame` and `Window`. Divides space into five geographical regions: `NORTH`, `SOUTH`, `EAST`, `WEST`, and `CENTER`.
- **GridLayout:** Places components in a uniform grid of equal-sized rows and columns.

## Quick Revision — Module 10

Event Handling & AWT | OE-EC704A

### Event & Layout Syntax

| Construct                | Syntax and Example Usage                                                            |
|--------------------------|-------------------------------------------------------------------------------------|
| Register Action Listener | <code>btn.addActionListener(listenerObj);</code>                                    |
| Anonymous Inner Handler  | <code>btn.addActionListener(new ActionListener() { ... });</code>                   |
| Set Frame Layout         | <code>frame.setLayout(new BorderLayout());</code>                                   |
| Add Component Layout     | <code>frame.add(btn, BorderLayout.CENTER);</code>                                   |
| Window Closing Adapter   | <code>f.addWindowListener(new WindowAdapter(){...WindowClosing...});</code>         |
| AWT Frame Init           | <code>Frame f = new Frame("Title"); f.setSize(300, 300); f.setVisible(true);</code> |

### One-Line Definitions

- **Delegation Event Model:** Framework routing UI events from sources to registered listener objects.
- **Event Source:** GUI control generating event objects on user state changes.
- **Event Listener:** Interface declaring abstract event handler callback operations.
- **Adapter Class:** Convenience class providing empty implementations of listener methods.
- **Heavyweight Control:** A UI component bound to and rendered by a native OS peer window.
- **AWT:** Heavyweight platform-dependent Java GUI framework package.
- **Layout Manager:** Object interface deciding component sizing and layouts within containers.
- **Anonymous Inner Class:** Unnamed class declared and instantiated inline at usage point.

### Frequently Asked / Viva Questions

#### Q1. How does the Delegation Event Model work?

An event source (like a button) generates an event object when interacted with, and delegates processing to one or more registered listener objects by invoking their callback methods.

#### Q2. What is the purpose of Adapter classes in AWT event handling?

**Frequently Asked / Viva Questions (contd.)**

They simplify listener implementations. Implementing interfaces like `WindowListener` requires writing empty overrides for all methods. Extending `WindowAdapter` allows overriding only the required methods.

**Q3. Why are AWT controls referred to as heavyweight components?**

AWT controls are coupled to the underlying operating system. Each AWT component is associated with a native peer window, consuming significant OS resources.

**Q4. What is the difference between AWT and Swing?**

AWT is platform-dependent and uses heavyweight OS peer rendering. Swing is platform-independent, lightweight, written entirely in Java, and supports a pluggable look-and-feel.

**Q5. How does `FlowLayout` arrange components? What is its default alignment?**

`FlowLayout` arranges components in a single line from left to right. When it runs out of space, it wraps them to the next line. Its default alignment is centered.

**Q6. What layout manager is default for `Frame`? `Panel`?**

The default layout manager for a `Frame` is `BorderLayout`. The default layout manager for a `Panel` (and `Applet`) is `FlowLayout`.

**Q7. Which listener interface is used to handle button clicks? Text changes?**

Button clicks generate `ActionEvents` and are handled by the `ActionListener` interface. Text changes in a `TextField` generate `TextEvents`, handled by the `TextListener` interface.

**Q8. How many regions does a `BorderLayout` have? What happens if no region is specified?**

Five regions: North, South, East, West, and Center. If no region is specified when adding a component, it is added to the `Center` region by default.

**Q9. Why is an anonymous inner class useful in event handling?**

It allows declaring and instantiating a listener class inline right where it is registered, keeping code localized and avoiding cluttering the outer class with simple single-method handlers.

**Q10. How can you close an AWT `Frame` window when the close button is clicked?**

By registering a `WindowListener` (or extending `WindowAdapter`) and calling `System.exit(0)` or `dispose()` inside the overridden `windowClosing(WindowEvent e)` method.

**Comparison: AWT vs. Swing**

| Metric           | AWT (Heavyweight)                | Swing (Lightweight)                 |
|------------------|----------------------------------|-------------------------------------|
| Rendering Engine | OS Native Peer controls.         | Painted in Java bytecode.           |
| Platform Styling | Inconsistent look-and-feel.      | Uniform style across OS targets.    |
| Class Packages   | Lives in <code>java.awt.*</code> | Lives in <code>javax.swing.*</code> |
| GUI Speed        | Slightly faster (OS optimized).  | Slightly slower (Java painted).     |